PRAXISHANDBUCH

Linux Security

Band 1: Grundlagen

Linux-Sicherheitsmechanismen verstehen und selbstständig umsetzen



VERSTEHEN, WARUM. NICHT NUR, WIE MAN EINEN BEFEHL EINTIPPT.

Mit 20 Laborübungen, Kontrollfragen und Lernzielkontrollen · Ausgabe Oktober 2026

Linux Security
Band 1: Grundlagen

Linux-Sicherheitsmechanismen verstehen und selbstständig umsetzen

Von Benutzerverwaltung über SSH und Firewall bis zu Protokollierung und Verschlüsselung. Mit 20 Laborübungen auf Debian 13, Kontrollfragen und Lernzielkontrollen je Kapitel.

Inhaltsverzeichnis

Vorwort 6

Zielgruppe und Lernweg 6

Die Laborumgebung dieses Bandes 6

Installation der Laborumgebung 7

Rücksetzung der Laborumgebung 7

Hinweis zu Quellen und Aktualität 7

Kapitel 1: Einführung in Linux Security 8

1. Einführung 8

2. Technische Grundlagen 8

3. Fachbegriffe 9

4. Praxis und Konfiguration 9

5. Sicherheitsbetrachtung 9

6. Praktische Laborübungen 9

7. Typische Fehler 10

8. Zusammenfassung 10

9. Kontrollfragen 11

10. Lernzielkontrolle 11

Kapitel 2: Linux-Systemarchitektur 12

1. Einführung 12

2. Technische Grundlagen 12

3. Fachbegriffe 12

4. Praxis und Konfiguration 13

5. Sicherheitsbetrachtung 13

6. Praktische Laborübungen 14

7. Typische Fehler 14

8. Zusammenfassung 15

9. Kontrollfragen 15

10. Lernzielkontrolle 15

Kapitel 3: Benutzer, Gruppen und Berechtigungen 16

1. Einführung 16

2. Technische Grundlagen 16

3. Fachbegriffe 17

4. Praxis und Konfiguration . . . 17
5. Sicherheitsbetrachtung . . . 17
6. Praktische Laborübungen . . . 18
7. Typische Fehler . . . 18
8. Zusammenfassung . . . 18
9. Kontrollfragen . . . 18
10. Lernzielkontrolle . . . 19

Kapitel 4: Authentifizierung und Zugriffsschutz . . . 20

1. Einführung . . . 20
2. Technische Grundlagen . . . 20
3. Fachbegriffe . . . 20
4. Praxis und Konfiguration . . . 21
5. Sicherheitsbetrachtung . . . 21
6. Praktische Laborübungen . . . 22
7. Typische Fehler . . . 22
8. Zusammenfassung . . . 23
9. Kontrollfragen . . . 23
10. Lernzielkontrolle . . . 23

Kapitel 5: SSH-Sicherheit . . . 24

1. Einführung . . . 24
2. Technische Grundlagen . . . 24
3. Fachbegriffe . . . 24
4. Praxis und Konfiguration . . . 24
5. Sicherheitsbetrachtung . . . 25
6. Praktische Laborübungen . . . 26
7. Typische Fehler . . . 26
8. Zusammenfassung . . . 26
9. Kontrollfragen . . . 27
10. Lernzielkontrolle . . . 27

Kapitel 6: Grundlagen der Netzwerksicherheit . . . 28

1. Einführung . . . 28
2. Technische Grundlagen . . . 28
3. Fachbegriffe . . . 28
4. Praxis und Konfiguration . . . 29
5. Sicherheitsbetrachtung . . . 29

6. Praktische Laborübungen . . . 29
7. Typische Fehler . . . 30
8. Zusammenfassung . . . 30
9. Kontrollfragen . . . 30
10. Lernzielkontrolle . . . 31

Kapitel 7: Firewall-Grundlagen . . . 32

1. Einführung . . . 32
2. Technische Grundlagen . . . 32
3. Fachbegriffe . . . 32
4. Praxis und Konfiguration . . . 33
5. Sicherheitsbetrachtung . . . 33
6. Praktische Laborübungen . . . 34
7. Typische Fehler . . . 34
8. Zusammenfassung . . . 34
9. Kontrollfragen . . . 35
10. Lernzielkontrolle . . . 35

Kapitel 8: Sicherheitsupdates und Paketverwaltung . . . 36

1. Einführung . . . 36
2. Technische Grundlagen . . . 36
3. Fachbegriffe . . . 36
4. Praxis und Konfiguration . . . 36
5. Sicherheitsbetrachtung . . . 37
6. Praktische Laborübungen . . . 37
7. Typische Fehler . . . 38
8. Zusammenfassung . . . 38
9. Kontrollfragen . . . 38
10. Lernzielkontrolle . . . 39

Kapitel 9: Linux-Protokollierung und Systemüberwachung . . . 40

1. Einführung . . . 40
2. Technische Grundlagen . . . 40
3. Fachbegriffe . . . 40
4. Praxis und Konfiguration . . . 40
5. Sicherheitsbetrachtung . . . 41
6. Praktische Laborübungen . . . 41
7. Typische Fehler . . . 42

8. Zusammenfassung . . . 42
9. Kontrollfragen . . . 42
10. Lernzielkontrolle . . . 43

Kapitel 10: Grundlagen der Verschlüsselung . . . 44

1. Einführung . . . 44
2. Technische Grundlagen . . . 44
3. Fachbegriffe . . . 44
4. Praxis und Konfiguration . . . 45
5. Sicherheitsbetrachtung . . . 45
6. Praktische Laborübungen . . . 45
7. Typische Fehler . . . 46
8. Zusammenfassung . . . 46
9. Kontrollfragen . . . 46
10. Lernzielkontrolle . . . 47
Abschlussprojekt Band 1: Sicherer Debian-Grundserver . . . 47

Verzeichnis der Befehlsbeispiele . . . 48

Index . . . 49

Vorwort

Dieses Buch ist der erste von drei Bänden einer Fachbuchreihe über Linux-Sicherheit. Band 1 vermittelt die Grundlagen: wie Linux-Systeme aufgebaut sind, wie Benutzer, Rechte und Authentifizierung zusammenspielen, wie ein System grundlegend gegen Netzwerkangriffe abgesichert wird, und wie Protokollierung und Verschlüsselung als letzte Sicherheitsnetze funktionieren.

Band 2 baut darauf auf und widmet sich der professionellen Härtung von Servern (nftables im Detail, AppArmor, Fail2ban, Auditierung, Container-Sicherheit). Band 3 behandelt Sicherheitsanalyse und ethisches Hacking (Nmap, Wireshark, Lynis, Metasploit, Incident Response, forensische Grundlagen). Alle drei Bände bauen aufeinander auf und nutzen dieselbe Laborumgebung.

Zielgruppe und Lernweg

Dieses Buch richtet sich an Leser:innen mit Grundkenntnissen der Linux-Kommandozeile, die systematisch verstehen wollen, warum bestimmte Sicherheitsmaßnahmen notwendig sind, nicht nur, wie man einen Befehl eintippt. Jedes Kapitel erklärt zuerst das Prinzip, dann die konkrete Umsetzung unter Debian 13.

Die Laborumgebung dieses Bandes

Für Band 1 genügen zwei virtuelle Maschinen:

System	Rolle	Betriebssystem
security-client	Arbeitsstation für Übungen, Schlüsselerzeugung, Verbindungstests	Kali Linux (aktuelle Version)
server1	Übungsserver, auf dem die meisten Konfigurationen vorgenommen werden	Debian 13 (Trixie)

Server 2 und ein Windows-11-Client kommen in Band 2 und 3 hinzu, sobald Netzwerksegmentierung und gemischte Umgebungen eine Rolle spielen.

Beispiel V.1: Netzwerkarchitektur dieses Bandes

```
security-client   192.168.56.10   (Kali Linux)
server1           192.168.56.20   (Debian 13)
Beide Systeme im selben, isolierten "Internal Network" einer
Virtualisierungssoftware (VirtualBox, VMware oder Hyper-V), ohne
Bridged-Adapter und ohne direkten Internetzugang für server1.
```

> **WARNUNG – Sicherheitsrisiko**
>
> Jede Übung in diesem Buch, die Konfigurationen verändert, Dienste neu startet oder Testangriffe simuliert, gehört ausschließlich in diese isolierte Laborumgebung. Vor jeder Übung empfiehlt sich ein Snapshot von `server1`, damit sich der Ausgangszustand jederzeit wiederherstellen lässt.

Installation der Laborumgebung

Beispiel V.2: server1 einrichten (Kurzübersicht)

```
1. Debian 13 Netinst-Image von debian.org herunterladen
2. Minimalinstallation ohne grafische Oberfläche, nur "SSH-Server"
   und "Standard-Systemwerkzeuge" auswählen
3. Netzwerkadapter der VM auf "Internes Netzwerk" stellen
4. Nach der Installation: $ sudo apt update && sudo apt full-upgrade
```

Beispiel V.3: security-client vorbereiten

```
1. Aktuelles Kali-Linux-Image von kali.org herunterladen
2. Netzwerkadapter ebenfalls auf dasselbe "Internes Netzwerk" stellen
3. $ ping -c 3 192.168.56.20   # Erreichbarkeit von server1 prüfen
```

Rücksetzung der Laborumgebung

Jedes Kapitel kann einzeln für sich geübt werden. Enthält ein Kapitel Konfigurationsänderungen, die eine spätere Übung beeinflussen könnten, weist die jeweilige Übung ausdrücklich darauf hin. Im Zweifel stellt ein Snapshot-Rücksprung auf `server1` einen sauberen Ausgangszustand wieder her.

Hinweis zu Quellen und Aktualität

Alle Befehle und Konfigurationsbeispiele in diesem Buch wurden gegen die offizielle Debian-Dokumentation und die jeweiligen Projektdokumentationen (OpenSSH, Linux-PAM, GNU) geprüft. Wo ein Verfahren als veraltet gilt, ist das ausdrücklich vermerkt. Dieses Buch ersetzt keine Rechtsberatung und keine offizielle Dokumentation; prüfe bei produktiven Systemen zusätzlich die jeweils aktuelle Quelle.

Kapitel 1: Einführung in Linux Security

Lernziele

- Du kannst erklären, warum Linux-Sicherheit ein eigenständiges Fachgebiet ist.
- Du ordnest ein Sicherheitsrisiko in die CIA-Triade ein.
- Du benennst die wichtigsten Angriffsflächen eines typischen Linux-Systems.

1. Einführung

Linux betreibt den überwiegenden Teil der Server im Internet, nahezu alle Supercomputer und die meiste eingebettete Infrastruktur, von Routern bis zu Steuerungsanlagen. Diese Verbreitung macht Linux-Systeme zu einem lohnenden Ziel, und sie macht Linux-Sicherheit zu einer Kernkompetenz jeder Fachkraft für Systemintegration. Wer versteht, wie Linux-Sicherheitsmechanismen ineinandergreifen, kann Systeme nicht nur nach einer Checkliste absichern, sondern begründete Entscheidungen treffen, wenn eine Checkliste nicht passt.

2. Technische Grundlagen

Sicherheit wird in der Fachliteratur meist über drei Schutzziele beschrieben, die sogenannte **CIA-Triade**:

Schutzziel	Bedeutung	Beispiel einer Verletzung
Confidentiality (Vertraulichkeit)	nur berechtigte Personen erhalten Zugriff auf Daten	ein unbefugt mitgelesenes Passwort
Integrity (Integrität)	Daten bleiben unverändert und nachvollziehbar	eine manipulierte Konfigurationsdatei
Availability (Verfügbarkeit)	Systeme und Daten stehen bei Bedarf zur Verfügung	ein durch Überlastung lahmgelegter Server

Jede Sicherheitsmaßnahme in diesem Buch lässt sich einem oder mehreren dieser drei Ziele zuordnen. Eine Firewall-Regel schützt primär Vertraulichkeit und Verfügbarkeit, eine Prüfsumme primär Integrität, eine Backup-Strategie primär Verfügbarkeit.

Die **Unix-Philosophie** prägt bis heute, wie Linux-Sicherheit gedacht wird: kleine, spezialisierte Werkzeuge, die eine Aufgabe gut erfüllen, kombiniert über einfache Schnittstellen. Daraus folgt ein Sicherheitsprinzip, das sich durch dieses ganze Buch zieht: das **Prinzip der geringsten Rechte** (Principle of Least Privilege). Ein Prozess, ein Dienst oder ein Benutzerkonto sollte immer nur genau die Rechte besitzen, die für seine Aufgabe nötig sind, nicht mehr.

3. Fachbegriffe

Ein **Exploit** ist ein Stück Code oder eine Befehlsfolge, die eine konkrete Schwachstelle gezielt ausnutzt. Eine **Schwachstelle** (Vulnerability) ist dagegen die zugrunde liegende Lücke selbst, unabhängig davon, ob bereits ein funktionierender Exploit dafür existiert. Ein **CVE** (Common Vulnerabilities and Exposures) ist eine eindeutige, öffentliche Kennung für eine bekannte Schwachstelle, etwa CVE-2024-12345, unter der sich Details in öffentlichen Datenbanken nachschlagen lassen.

Die **Angriffsfläche** eines Systems umfasst alles, worüber ein Angreifer potenziell Einfluss nehmen kann: offene Netzwerkports, installierte Software, Benutzerkonten, physische Zugänge. Ein **Härtungsmaßnahme** (Hardening) reduziert diese Angriffsfläche gezielt, etwa indem ein nicht benötigter Dienst deaktiviert wird.

4. Praxis und Konfiguration

Ein erster, praktischer Schritt ist, die eigene Angriffsfläche überhaupt sichtbar zu machen.

Beispiel 1.1: Installierte Pakete zählen

```
$ dpkg -l | wc -l
```

Beispiel 1.2: Laufende Netzwerkdienste auflisten

```
$ ss -tulpen
```

Beispiel 1.3: Aktuelle Debian-Version und Codename prüfen

```
$ cat /etc/os-release
```

Diese drei Befehle liefern noch keine Bewertung, aber die Rohdaten, auf denen jede Sicherheitsbetrachtung in den folgenden Kapiteln aufbaut.

5. Sicherheitsbetrachtung

> **WARNUNG – Sicherheitsrisiko**
>
> Ein frisch installiertes System gilt nicht automatisch als sicher. Die Standardinstallation von Debian ist bewusst zurückhaltend konfiguriert (wenige aktive Dienste), aber jede später installierte Software, jeder neu angelegte Benutzer und jede freigegebene Firewall-Regel vergrößert die Angriffsfläche erneut.

Typische, bereits auf Systemebene wirksame Risiken sind: unnötig installierte Pakete, die mit der Zeit veralten und ungeprüft bleiben; Standardkonten oder Testzugänge, die nach der Einrichtung nicht entfernt werden; und fehlende Prozesse, um neue Sicherheitslücken überhaupt zu bemerken (Kapitel 8 und 9 vertiefen das).

6. Praktische Laborübungen

Übung A: Die eigene Angriffsfläche dokumentieren

Aufgabenstellung: Erstelle eine kurze, schriftliche Bestandsaufnahme der Angriffsfläche von `server1`. **Ausgangssituation:** `server1` ist frisch nach Beispiel V.2 installiert. **Voraussetzungen:** SSH-Zugriff auf `server1` von `security-client` aus. **Schritt-für-Schritt-Anleitung und Befehle:**

Befehle zu Übung A

```
$ ssh benutzername@192.168.56.20
$ dpkg -l | wc -l
$ ss -tulpen
$ who
$ cat /etc/os-release
```

Erwartetes Ergebnis: Eine Liste installierter Pakete (Anzahl), eine kurze Liste laufender Netzwerkdienste (bei einer Minimalinstallation meist nur SSH auf Port 22), keine weiteren angemeldeten Benutzer außer dir selbst. **Überprüfung:** Vergleiche deine Notizen mit der Ausgabe von `ss -tulpen` – jeder dort gelistete Port sollte in deiner Dokumentation einen Eintrag haben, mit Begründung, warum der Dienst läuft. **Fehleranalyse:** Erscheinen unerwartete Dienste, prüfe mit `systemctl status <dienstname>`, wodurch sie automatisch gestartet wurden. **Musterlösung:** Eine Minimalinstallation zeigt typischerweise nur Port 22/tcp (SSH) als aktiv. Jeder zusätzliche Eintrag sollte hinterfragt werden.

Übung B: Ein aktuelles CVE einordnen

Aufgabenstellung: Suche im Debian Security Tracker ein CVE zu einem auf `server1` installierten Paket und ordne es nach CIA-Schutzziel ein. **Ausgangssituation:** Die Paketliste aus Übung A liegt vor. **Voraussetzungen:** Internetzugang von `security-client` aus (nicht von `server1`).
Schritt-für-Schritt-Anleitung:

Vorgehen zu Übung B

```
1. Wähle ein Paket aus deiner Liste, z. B. "openssh-server"
2. Öffne security-tracker.debian.org und suche nach diesem Paket
3. Notiere ein dort gelistetes CVE samt Kurzbeschreibung und Status
   (behoben, offen, "no-dsa" = kein eigenes Advisory nötig)
```

Erwartetes Ergebnis: Ein konkretes CVE mit Beschreibung und aktuellem Behebungsstatus für dein gewähltes Paket. **Überprüfung:** Lässt sich aus der Beschreibung ableiten, welches CIA-Schutzziel betroffen ist (z. B. Codeausführung aus der Ferne betrifft meist alle drei)? **Fehleranalyse:** Keine CVEs gefunden? Das ist für ein kleines, gut gepflegtes Paket ein normales, positives Ergebnis – wähle probeweise ein größeres Paket wie den Kernel. **Musterlösung:** Eine korrekte Lösung nennt CVE-Nummer, betroffenes Paket, eine Kurzbeschreibung in eigenen Worten und den aktuellen Status laut Tracker.

7. Typische Fehler

Fehler	Ursache	Behebung
„Das System ist neu, also sicher"	Verwechslung von Standardkonfiguration mit Absicherung	Angriffsfläche aktiv dokumentieren (Übung A), statt sie anzunehmen
CIA-Schutzziele nicht unterschieden	Sicherheit nur als „ein" Ziel gedacht	jede Maßnahme bewusst einem oder mehreren Zielen zuordnen
Härtung ohne Dokumentation	Änderungen werden vorgenommen, aber nicht festgehalten	jede Änderung mit Datum und Grund notieren (wird in Band 2 vertieft)

8. Zusammenfassung

Zusammenfassung

- Linux-Sicherheit stützt sich auf die drei Schutzziele Vertraulichkeit, Integrität und Verfügbarkeit.
- Das Prinzip der geringsten Rechte zieht sich durch alle folgenden Kapitel.
- CVEs sind eindeutige, öffentliche Kennungen bekannter Schwachstellen.
- Die eigene Angriffsfläche lässt sich mit einfachen Bordmitteln (`dpkg`, `ss`) sichtbar machen.

9. Kontrollfragen

1. Welche drei Schutzziele bilden die CIA-Triade? (leicht)
2. Was unterscheidet eine Schwachstelle von einem Exploit? (leicht)
3. Warum gilt eine Standardinstallation nicht automatisch als sicher? (mittel)
4. Nenne zwei Befehle, mit denen sich die Angriffsfläche eines Systems grob erfassen lässt. (mittel)
5. Ein Server führt einen Dienst aus, der nicht gebraucht wird. Welchem Schutzziel nützt es am meisten, ihn zu deaktivieren, und warum? (schwer)

Lösungshinweise: 1. Confidentiality, Integrity, Availability. 2. Die Schwachstelle ist die Lücke selbst, der Exploit nutzt sie aktiv aus. 3. Weil jede später installierte Software und jedes neue Konto die Angriffsfläche erneut vergrößert. 4. Zum Beispiel `dpkg -l` und `ss -tulpen`. 5. Vor allem Confidentiality und Availability, da ein ungenutzter, aber laufender Dienst ein zusätzliches, unnötiges Einfallstor darstellt, ohne einen Nutzen zu bieten, der das Risiko aufwiegt.

10. Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] die CIA-Triade ohne Nachschlagen erklären kannst,
- [ ] mit `dpkg` und `ss` die grobe Angriffsfläche eines Systems selbst erfassen kannst,
- [ ] zu einem selbst gewählten Paket ein reales CVE im Debian Security Tracker gefunden hast,
- [ ] erklären kannst, warum das Prinzip der geringsten Rechte für die folgenden Kapitel wichtig wird.

Kapitel 2: Linux-Systemarchitektur

Lernziele

- Du erklärst den Unterschied zwischen Kernel- und Userspace und seine sicherheitsrelevante Bedeutung.
- Du liest die Rechte eines laufenden Prozesses aus.
- Du ordnest die wichtigsten Verzeichnisse der Dateisystemhierarchie sicherheitsrelevant ein.

1. Einführung

Bevor sich einzelne Sicherheitsmechanismen sinnvoll erklären lassen, braucht es ein Grundverständnis dafür, wie ein laufendes Linux-System überhaupt aufgebaut ist. Wer weiß, welcher Teil des Systems mit welchen Rechten läuft, versteht auch, warum eine Schwachstelle an der einen Stelle harmlos und an einer anderen kritisch ist.

2. Technische Grundlagen

Linux trennt strikt zwischen **Kernelspace** und **Userspace**. Der Kernel verwaltet Hardware, Speicher und Prozesse mit uneingeschränkten Rechten; Anwendungsprogramme laufen im Userspace mit eingeschränkten Rechten und müssen für privilegierte Operationen (Dateizugriff, Netzwerkkommunikation) über definierte Schnittstellen, die **Systemaufrufe** (Syscalls), beim Kernel anfragen.

Beispiel 2.1: Die grobe Architektur (Konzept)

```
Anwendungen (Userspace)
    |  Systemaufrufe (Syscalls)
Kernel (Kernelspace)
    |  Gerätetreiber
Hardware
```

Seit Debian 10 nutzt Debian standardmäßig **systemd** als Init-System – den ersten Prozess, der beim Systemstart läuft (PID 1) und alle weiteren Dienste startet und überwacht. systemd organisiert Dienste in **Units**, die wichtigste Art davon sind **Service-Units**.

Beispiel 2.2: Eine systemd-Service-Unit anzeigen

```
$ systemctl status ssh.service
```

Jede Unit lässt sich einzeln konfigurieren, auch sicherheitsrelevant: systemd erlaubt, einem Dienst gezielt Rechte zu entziehen (zum Beispiel keinen Zugriff auf `/home` zu erlauben), ohne den Dienst selbst zu verändern – ein praktisches Beispiel des Prinzips der geringsten Rechte aus Kapitel 1.

3. Fachbegriffe

Ein **Prozess** ist eine laufende Instanz eines Programms, mit einer eigenen Prozess-ID (PID) und eigenen Rechten, die sich von denen anderer Prozesse unterscheiden können. Die **Dateisystemhierarchie** (Filesystem Hierarchy Standard, FHS) legt fest, wofür Standardverzeichnisse wie `/etc`, `/var` oder `/home` vorgesehen sind – ein Angreifer, der die FHS kennt, weiß auch, wo typischerweise Konfigurationsdateien, Protokolle oder Benutzerdaten liegen.

Ein **Target** ist in systemd eine Gruppe von Units, die gemeinsam einen bestimmten Systemzustand beschreiben, etwa `multi-user.target` für ein vollständig hochgefahrenes Mehrbenutzersystem ohne grafische Oberfläche. **Stable** bezeichnet den Debian-Veröffentlichungszweig, der nach ausführlichem Testen über einen langen Zeitraum ohne größere Funktionsänderungen gepflegt wird – der Gegensatz dazu sind Rolling-Release-Distributionen, die neue Softwareversionen fortlaufend statt in festen Zyklen einspielen.

4. Praxis und Konfiguration

Beispiel 2.3: Rechte eines laufenden Prozesses anzeigen

```
$ ps -eo pid,user,cmd | grep sshd
```

Beispiel 2.4: Alle aktiven systemd-Units eines Typs auflisten

```
$ systemctl list-units --type=service --state=running
```

Beispiel 2.5: Eine eigene, einfache systemd-Unit anlegen

```
$ sudo nano /etc/systemd/system/beispiel.service
```

Beispiel 2.6: Inhalt von beispiel.service

```
[Unit]
Description=Beispieldienst für Kapitel 2

[Service]
ExecStart=/usr/bin/sleep infinity
User=nobody

[Install]
WantedBy=multi-user.target
```

`User=nobody` sorgt dafür, dass der Dienst nicht mit root-Rechten läuft, obwohl er von root aus eingerichtet wurde – ein einfaches, aber wirkungsvolles Beispiel des Prinzips der geringsten Rechte direkt in der systemd-Konfiguration.

5. Sicherheitsbetrachtung

> **WARNUNG – Sicherheitsrisiko**
>
> Ein Dienst, der unnötig als root läuft, vervielfacht den Schaden einer einzelnen Schwachstelle in diesem Dienst: Ein Angreifer, der über eine Lücke im Dienst Code ausführen kann, erhält dann sofort volle Systemrechte statt eingeschränkter Rechte. Prüfe für jeden selbst eingerichteten Dienst, ob er tatsächlich root-Rechte benötigt.

Die FHS selbst ist keine Sicherheitsmaßnahme, hat aber sicherheitsrelevante Konsequenzen: `/tmp` ist für alle Benutzer beschreibbar und wird deshalb oft als Zwischenstation bei Angriffen missbraucht; `/etc` enthält die meisten Konfigurationsdateien und sollte ausschließlich für root beschreibbar sein; `/var/log` enthält Protokolle, deren Integrität für eine spätere Analyse entscheidend ist.

6. Praktische Laborübungen

Übung A: Prozessrechte systematisch untersuchen

Aufgabenstellung: Ermittle für drei laufende Dienste auf `server1`, mit welchen Rechten sie tatsächlich laufen. **Ausgangssituation:** `server1` läuft mit Standarddiensten aus der Minimalinstallation. **Voraussetzungen:** SSH-Zugriff auf `server1`. **Schritt-für-Schritt-Anleitung:**

Befehle zu Übung A

```
$ ps -eo pid,user,cmd | head -20
$ systemctl status ssh.service
$ systemctl status systemd-journald.service
```

Erwartetes Ergebnis: `sshd` läuft typischerweise mit einem Elternprozess als root (für das Binden an Port 22), bedient einzelne Verbindungen aber mit den Rechten des sich anmeldenden Benutzers. `systemd-journald` läuft mit einem eigenen, eingeschränkten Systemkonto. **Überprüfung:** Kein hier geprüfter Dienst sollte dauerhaft und vollständig als `root` laufen, ohne dass du eine Begründung dafür findest. **Fehleranalyse:** Läuft ein Dienst unerwartet als root, prüfe seine `.service`-Datei unter `/etc/systemd/system/` oder `/lib/systemd/system/` auf eine `User=`-Zeile. **Musterlösung:** Eine vollständige Lösung listet für alle drei Dienste den tatsächlich ausführenden Benutzer und erklärt in einem Satz, warum dieser Benutzer angemessen ist.

Übung B: Eine eigene, rechtebeschränkte systemd-Unit anlegen

Aufgabenstellung: Lege den Dienst aus Beispiel 2.5/2.6 an, starte ihn, und verifiziere seine Rechte. **Ausgangssituation:** Noch keine eigene Unit vorhanden. **Voraussetzungen:** sudo-Rechte auf `server1`. **Schritt-für-Schritt-Anleitung:**

Befehle zu Übung B

```
$ sudo systemctl daemon-reload
$ sudo systemctl start beispiel.service
$ systemctl status beispiel.service
$ ps -eo pid,user,cmd | grep sleep
```

Erwartetes Ergebnis: Der Dienst läuft, der Prozess `sleep infinity` erscheint mit dem Benutzer `nobody`, nicht `root`. **Überprüfung:** `ps` zeigt in der User-Spalte `nobody`. **Fehleranalyse:** Zeigt `ps` dennoch `root`, prüfe, ob `daemon-reload` tatsächlich ausgeführt wurde und ob die Datei fehlerfrei gespeichert ist (`systemctl status` zeigt Parserfehler an). **Musterlösung:** Nach erfolgreichem Start zeigt `systemctl status beispiel.service` „active (running)“ und `ps` bestätigt den Benutzer `nobody`. Beende die Übung mit `sudo systemctl stop beispiel.service` und `sudo systemctl disable beispiel.service`, um den Ausgangszustand wiederherzustellen.

7. Typische Fehler

Fehler	Ursache	Behebung
Eigener Dienst läuft unnötig als root	keine `User=`-Zeile in der Unit gesetzt	passenden, eingeschränkten Benutzer ergänzen
Änderung an `.service`-Datei wirkt nicht	`daemon-reload` vergessen	nach jeder Änderung `systemctl daemon-reload` ausführen
Verwechslung von Target und Service	beide als „irgendeine Unit“ behandelt	Target = Zustand/Gruppe, Service = einzelner Dienst

8. Zusammenfassung

Zusammenfassung

- Kernelspace hat uneingeschränkte, Userspace eingeschränkte Rechte; Systemaufrufe verbinden beide kontrolliert.
- systemd verwaltet Dienste als Units und erlaubt, ihnen gezielt Rechte zu entziehen.
- Die Dateisystemhierarchie hat sicherheitsrelevante Konsequenzen, besonders bei `/tmp`, `/etc` und `/var/log`.
- Eigene Dienste sollten nie ohne Grund mit root-Rechten laufen.

9. Kontrollfragen

1. Was unterscheidet Kernelspace von Userspace? (leicht)
2. Wofür steht die Abkürzung PID? (leicht)
3. Warum ist `/tmp` aus Sicherheitssicht ein besonderes Verzeichnis? (mittel)
4. Wie entzieht man einem systemd-Dienst gezielt root-Rechte? (mittel)
5. Ein Dienst benötigt laut Dokumentation Zugriff auf Port 80, der unter 1024 liegt und traditionell root-Rechte voraussetzt. Wie lässt sich das heute lösen, ohne den gesamten Dienst dauerhaft als root laufen zu lassen? (schwer)

Lösungshinweise: 1. Kernelspace hat vollen Hardware-/Systemzugriff, Userspace läuft eingeschränkt und muss über Syscalls anfragen. 2. Process ID. 3. Weil es für alle Benutzer beschreibbar ist und oft als Zwischenstation für Angriffe dient. 4. Über die Option `User=` (und `Group=`) in der systemd-Service-Datei. 5. Über systemd-Capabilities (`AmbientCapabilities=CAP_NET_BIND_SERVICE` in der Unit) lässt sich gezielt nur die eine benötigte Fähigkeit vergeben, ohne vollständige root-Rechte zu gewähren.

10. Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] den Unterschied zwischen Kernel- und Userspace in eigenen Worten erklären kannst,
- [ ] mit `ps` und `systemctl status` die Rechte eines laufenden Dienstes ermittelst,
- [ ] eine eigene systemd-Unit mit eingeschränktem Benutzer anlegen und starten kannst,
- [ ] erklären kannst, warum `/tmp`, `/etc` und `/var/log` sicherheitsrelevant unterschiedlich zu behandeln sind.

Kapitel 3: Benutzer, Gruppen und Berechtigungen

Lernziele

- Du legst Benutzer und Gruppen korrekt an und ordnest sie zu.
- Du liest und setzt klassische Dateirechte sowie ACLs.
- Du unterscheidest `su` und `sudo` und konfigurierst eine einfache `sudoers`-Regel.

1. Einführung

Fast jede Sicherheitsentscheidung unter Linux läuft letztlich auf eine Frage hinaus: Welches Konto darf was? Dieses Kapitel legt das Fundament, auf dem die Authentifizierung (Kapitel 4), SSH-Zugriff (Kapitel 5) und die spätere Rechteausweitungs-Thematik aus Band 3 aufbauen.

2. Technische Grundlagen

Jeder Benutzer besitzt eine numerische **UID** (User ID), jede Gruppe eine **GID** (Group ID). Das Konto `root` trägt immer die UID 0 und besitzt uneingeschränkte Rechte. Benutzerinformationen stehen in `/etc/passwd` (öffentlich lesbar), die eigentlichen Passwort-Hashes dagegen in `/etc/shadow` (nur für root lesbar) – eine Trennung, die historisch gewachsen ist, um zu verhindern, dass jedes Programm, das Benutzernamen nachschlagen muss, automatisch auch Zugriff auf Passwort-Hashes bekommt.

Beispiel 3.1: Einen Eintrag aus /etc/passwd lesen

```
eugen:x:1000:1000:Eugen Becker:/home/eugen:/bin/bash
```

Feld	Bedeutung
`eugen`	Benutzername
`x`	Platzhalter, echter Hash steht in `/etc/shadow`
`1000`	UID
`1000`	primäre GID
`Eugen Becker`	Kommentarfeld (GECOS)
`/home/eugen`	Home-Verzeichnis
`/bin/bash`	Standard-Shell

Jede Datei trägt drei Rechteblöcke (Eigentümer, Gruppe, Andere) mit je lesen, schreiben, ausführen – dasselbe Grundmodell, das auch im weiterführenden Teil dieser Buchreihe zu Linux Privilege Escalation vertieft wird. **ACLs** (Access Control Lists) erweitern dieses einfache Modell um Rechte für zusätzliche, einzeln benannte Benutzer oder Gruppen, wenn die klassischen drei Blöcke nicht ausreichen.

3. Fachbegriffe

sudo erlaubt einem berechtigten Benutzer, einzelne Befehle mit den Rechten eines anderen Kontos (meist root) auszuführen, protokolliert dabei jeden Aufruf und verlangt in der Standardeinstellung das eigene Passwort, nicht das von root. **su** wechselt dagegen vollständig zu einem anderen Konto und verlangt dessen Passwort. Die Datei `/etc/sudoers`, bearbeitet ausschließlich mit `visudo`, legt fest, wer welche Befehle als wer ausführen darf.

Eine **ACL** fügt einer Datei zusätzliche Rechteeinträge hinzu, ohne die klassischen Unix-Rechte zu ersetzen; `getfacl` liest, `setfacl` setzt sie. Das **SUID-Bit** lässt ein Programm mit den Rechten seines Eigentümers statt der ausführenden Person laufen – ein mächtiger, aber auch missbrauchsanfälliger Mechanismus, der in Band 3 dieser Reihe vertieft behandelt wird.

4. Praxis und Konfiguration

Beispiel 3.2: Benutzer und Gruppe anlegen

```
$ sudo groupadd projekt
$ sudo useradd -m -g projekt -s /bin/bash testuser
$ sudo passwd testuser
```

Beispiel 3.3: Klassische Rechte setzen und lesen

```
$ chmod 750 /home/testuser/daten
$ ls -ld /home/testuser/daten
drwxr-x--- 2 testuser projekt 4096 Jan 10 09:00 /home/testuser/daten
```

Beispiel 3.4: Eine ACL ergänzen

```
$ sudo setfacl -m u:eugen:rx /home/testuser/daten
$ getfacl /home/testuser/daten
```

Beispiel 3.5: Eine einfache sudoers-Regel (mit visudo bearbeiten)

```
testuser ALL=(root) /usr/bin/systemctl restart apache2
```

> **WARNUNG – Sicherheitsrisiko**
>
> Bearbeite `/etc/sudoers` ausschließlich mit `visudo`, niemals mit einem gewöhnlichen Editor direkt. `visudo` prüft die Syntax vor dem Speichern; ein Syntaxfehler in einer direkt gespeicherten Datei kann dazu führen, dass `sudo` systemweit nicht mehr funktioniert.

5. Sicherheitsbetrachtung

Eine zu großzügige `sudoers`-Regel (zum Beispiel ein Platzhalter wie `ALL=(ALL) ALL` für ein Konto, das nur einen einzigen Dienst neu starten muss) vergrößert die Angriffsfläche unnötig: Wird dieses Konto kompromittiert, erhält ein Angreifer damit praktisch root-Rechte. Dieselbe Überlegung gilt für ACLs: Jedes zusätzlich vergebene Recht sollte einen konkreten, dokumentierten Grund haben.

6. Praktische Laborübungen

Übung A: Benutzer, Gruppe und Rechte gezielt setzen

Aufgabenstellung: Lege einen Benutzer und eine Gruppe an und richte ein Verzeichnis mit passenden Rechten ein. **Ausgangssituation:** `server1`, Standardkonfiguration. **Voraussetzungen:** sudo-Rechte. **Schritt-für-Schritt-Anleitung:** siehe Beispiel 3.2 und 3.3. **Erwartetes Ergebnis:** `ls -ld` zeigt `drwxr-x---`, Eigentümer `testuser`, Gruppe `projekt`. **Überprüfung:** Ein Mitglied der Gruppe `projekt` kann das Verzeichnis lesen und betreten, ein fremdes Konto nicht. **Fehleranalyse:** Falsche Rechte? Prüfe mit `id testuser`, ob die primäre Gruppe tatsächlich `projekt` ist. **Musterlösung:** `drwxr-x--- 2 testuser projekt ... daten` entspricht der Vorgabe aus Beispiel 3.3.

Übung B: Eine ACL anwenden und verifizieren

Aufgabenstellung: Gib einem dritten Benutzer über eine ACL Lesezugriff auf das Verzeichnis aus Übung A, ohne die Gruppenrechte zu verändern. **Ausgangssituation:** Ergebnis aus Übung A. **Voraussetzungen:** Paket `acl` installiert (`sudo apt install acl`). **Schritt-für-Schritt-Anleitung:** siehe Beispiel 3.4. **Erwartetes Ergebnis:** `getfacl` zeigt einen zusätzlichen Eintrag `user:eugen:r-x`. **Überprüfung:** Der Benutzer `eugen` kann das Verzeichnis lesen, obwohl er nicht Mitglied der Gruppe `projekt` ist. **Fehleranalyse:** Kein zusätzlicher Zugriff? Prüfe, ob das Dateisystem ACLs unterstützt (bei Standard-ext4 unter Debian 13 ist das der Fall) und ob `setfacl` tatsächlich fehlerfrei durchlief. **Musterlösung:** `getfacl` zeigt sowohl die klassischen Unix-Rechte als auch den zusätzlichen ACL-Eintrag `user:eugen:r-x` nebeneinander.

7. Typische Fehler

Fehler	Ursache	Behebung
`/etc/sudoers` direkt mit einem Editor geöffnet	Unkenntnis der `visudo`-Konvention	immer `visudo` nutzen
ACL gesetzt, aber wirkungslos	Dateisystem unterstützt keine ACLs, oder Paket `acl` fehlt	Dateisystemtyp und Paketinstallation prüfen
`sudo`-Regel zu breit formuliert	Bequemlichkeit statt konkreter Eingrenzung	Regel auf den tatsächlich benötigten Befehl beschränken

8. Zusammenfassung

Zusammenfassung

- UID/GID und die Trennung von `/etc/passwd` und `/etc/shadow` bilden das Fundament der Benutzerverwaltung.
- Klassische Unix-Rechte (Eigentümer, Gruppe, Andere) lassen sich durch ACLs gezielt erweitern.
- `sudo` protokolliert und beschränkt, `su` wechselt vollständig das Konto.
- `/etc/sudoers` wird ausschließlich mit `visudo` bearbeitet.

9. Kontrollfragen

1. Was bedeuten UID und GID? (leicht)
2. Warum liegen Passwort-Hashes in `/etc/shadow` statt in `/etc/passwd`? (leicht)

3. Was ist der praktische Unterschied zwischen `su` und `sudo`? (mittel)
4. Wie erweitert eine ACL die klassischen Unix-Dateirechte? (mittel)
5. Ein Konto braucht laut Anforderung „volle administrative Rechte für Wartungsarbeiten“. Welche Rückfrage solltest du vor der Konfiguration stellen, und warum? (schwer)

Lösungshinweise: 1. User ID und Group ID, numerische Kennungen für Benutzer und Gruppen. 2. Damit nicht jedes Programm, das `/etc/passwd` lesen darf, automatisch Zugriff auf Passwort-Hashes hat. 3. `su` wechselt vollständig das Konto und verlangt dessen Passwort, `sudo` führt einzelne Befehle mit fremden Rechten aus und verlangt das eigene Passwort. 4. Sie erlaubt zusätzliche, einzeln benannte Rechteeinträge für weitere Benutzer oder Gruppen, über die drei klassischen Blöcke hinaus. 5. Welche konkreten Befehle tatsächlich gebraucht werden, um die Regel so eng wie möglich statt pauschal mit `ALL=(ALL) ALL` zu formulieren und damit das Risiko einer kompromittierten Zugangsdaten zu begrenzen.

10. Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] einen Benutzer samt Gruppe korrekt anlegen kannst,
- [ ] Dateirechte mit `chmod` setzen und mit `ls -l` lesen kannst,
- [ ] eine ACL mit `setfacl` setzen und mit `getfacl` prüfen kannst,
- [ ] eine eng begrenzte `sudoers`-Regel mit `visudo` formulieren kannst.

Kapitel 4: Authentifizierung und Zugriffsschutz

Lernziele

- Du erklärst die Rolle von PAM im Authentifizierungsablauf.
- Du konfigurierst eine Passwort-Richtlinie mit `pam_pwquality`.
- Du richtest eine Kontosperrung nach Fehlversuchen mit `pam_faillock` ein.

1. Einführung

Benutzerkonten und Dateirechte aus Kapitel 3 setzen voraus, dass sich wirklich die berechtigte Person anmeldet. Dieses Kapitel behandelt, wie Linux diese Prüfung organisiert, wie sich Passwort-Richtlinien technisch durchsetzen lassen, und wie ein System auf wiederholte Fehlversuche reagiert.

2. Technische Grundlagen

PAM (Pluggable Authentication Modules) ist die zentrale Schicht, über die praktisch jeder Anmeldevorgang unter Linux läuft, von der lokalen Konsole über SSH bis zu `sudo`. PAM besteht aus austauschbaren Modulen, die in einer festgelegten Reihenfolge geprüft werden – eine Anwendung muss selbst nicht wissen, ob am Ende ein Passwort, ein Hardware-Token oder ein biometrisches Verfahren über Erfolg oder Misserfolg entscheidet.

Beispiel 4.1: Grundstruktur einer PAM-Konfiguration (Auszug aus /etc/pam.d/common-auth)

```
auth [success=1 default=ignore] pam_unix.so nullok
auth requisite    pam_deny.so
auth required     pam_permit.so
```

Jede Zeile ruft ein Modul auf und legt fest, wie das Ergebnis dieses Moduls den Gesamtablauf beeinflusst (`required`, `requisite`, `sufficient`, `optional`). Diese Struktur erlaubt, zusätzliche Prüfungen wie eine Passwort-Richtlinie oder eine Kontosperrung einzufügen, ohne die Grundauthentifizierung selbst zu verändern.

3. Fachbegriffe

Authentifizierung beantwortet die Frage „bist du, wer du vorgibst zu sein?“, meist über Passwort, Schlüssel oder beides. **Autorisierung** beantwortet die davon unabhängige Frage „darfst du das, was du gerade versuchst?“ – ein authentifizierter Benutzer kann trotzdem für eine bestimmte Aktion nicht autorisiert sein, wie die Rechteprüfungen aus Kapitel 3 zeigen.

pam_pwquality prüft neue Passwörter gegen konfigurierbare Qualitätsregeln (Mindestlänge, Zeichenvielfalt), bevor sie akzeptiert werden. **pam_faillock** zählt fehlgeschlagene Anmeldeversuche pro Konto und sperrt es nach einer konfigurierbaren Anzahl für eine bestimmte Zeit. **TOTP** (Time-based One-Time Password) erzeugt zeitlich begrenzt gültige, sechsstellige Codes als zweiten Faktor, zusätzlich zum Passwort.

4. Praxis und Konfiguration

Beispiel 4.2: pam_pwquality installieren und konfigurieren

```
$ sudo apt install libpam-pwquality
$ sudo nano /etc/security/pwquality.conf
```

Beispiel 4.3: Auszug aus pwquality.conf

```
minlen = 12
dcredit = -1
ucredit = -1
lcredit = -1
ocredit = -1
```

Parameter	Bedeutung
`minlen`	Mindestlänge des Passworts
`dcredit`	negativer Wert erzwingt mindestens eine Ziffer
`ucredit`	negativer Wert erzwingt mindestens einen Großbuchstaben
`lcredit`	negativer Wert erzwingt mindestens einen Kleinbuchstaben
`ocredit`	negativer Wert erzwingt mindestens ein Sonderzeichen

Beispiel 4.4: pam_faillock aktivieren (Auszug aus /etc/pam.d/common-auth)

```
auth    required                        pam_faillock.so preauth
auth    [success=1 default=ignore]      pam_unix.so nullok
auth    [default=die]                   pam_faillock.so authfail
auth    requisite                       pam_deny.so
auth    required                        pam_permit.so
auth    required                        pam_faillock.so authsucc
```

Beispiel 4.5: faillock-Verhalten konfigurieren (/etc/security/faillock.conf)

```
deny = 5
unlock_time = 600
```

Nach fünf Fehlversuchen sperrt dieses Beispiel das betroffene Konto für 600 Sekunden (zehn Minuten).

5. Sicherheitsbetrachtung

> **WARNUNG – Sicherheitsrisiko**
>
> Teste jede Änderung an PAM-Konfigurationsdateien zunächst in einer zusätzlichen, bereits angemeldeten Sitzung, bevor du die aktuelle Sitzung schließt. Ein Fehler in der PAM-Konfiguration kann dazu führen, dass sich niemand mehr anmelden kann, auch nicht root – ohne eine zweite, noch offene Sitzung bleibt dann nur der Zugriff über die Konsole eines Hypervisors.

Eine zu niedrige `deny`-Schwelle bei `pam_faillock` eröffnet selbst einen Denial-of-Service-Weg: Ein Angreifer könnte gezielt falsche Passwörter für ein fremdes Konto eingeben, um es zu sperren. Eine Abwägung zwischen Schutz vor Brute-Force und diesem Risiko ist Teil jeder sinnvollen Konfiguration.

6. Praktische Laborübungen

Übung A: Passwort-Richtlinie einrichten und testen

Aufgabenstellung: Richte `pam_pwquality` nach Beispiel 4.2/4.3 ein und teste sie mit einem absichtlich schwachen Passwort. **Ausgangssituation:** `server1`, PAM in Standardkonfiguration. **Voraussetzungen:** sudo-Rechte, eine zweite, offene SSH-Sitzung als Sicherheitsnetz. **Schritt-für-Schritt-Anleitung:**

Befehle zu Übung A

```
$ sudo apt install libpam-pwquality
$ sudo nano /etc/security/pwquality.conf   # Werte aus Beispiel 4.3 eintragen
$ sudo passwd testuser                      # mit Passwort "test" versuchen
```

Erwartetes Ergebnis: Das Passwort „test“ wird als zu kurz und zu einfach abgelehnt; ein Passwort wie `Tr4kt0r!Werk2026` wird akzeptiert. **Überprüfung:** Die Fehlermeldung von `passwd` nennt den konkreten Grund der Ablehnung. **Fehleranalyse:** Wird jedes Passwort akzeptiert? Prüfe, ob `libpam-pwquality` tatsächlich installiert und `pwquality.conf` korrekt gespeichert ist. **Musterlösung:** Nach korrekter Konfiguration meldet `passwd` bei „test“ einen Fehler wie „BAD PASSWORD: it is too short“.

Übung B: Kontosperrung nach Fehlversuchen auslösen

Aufgabenstellung: Richte `pam_faillock` nach Beispiel 4.4/4.5 ein und löse eine Sperrung absichtlich aus. **Ausgangssituation:** Ergebnis aus Übung A, zweite offene Sitzung weiterhin vorhanden. **Voraussetzungen:** sudo-Rechte. **Schritt-für-Schritt-Anleitung:**

Befehle zu Übung B

```
$ sudo nano /etc/pam.d/common-auth       # Zeilen aus Beispiel 4.4 ergänzen
$ sudo nano /etc/security/faillock.conf  # Werte aus Beispiel 4.5 eintragen
# Von einem dritten Terminal aus: 5x absichtlich falsches Passwort für testuser eingeben
$ faillock --user testuser
```

Erwartetes Ergebnis: `faillock --user testuser` zeigt nach dem fünften Fehlversuch mehrere protokollierte, fehlgeschlagene Versuche und einen aktiven Sperrzustand. **Überprüfung:** Ein sechster Versuch mit dem korrekten Passwort schlägt ebenfalls fehl, solange die Sperrzeit läuft. **Fehleranalyse:** Keine Sperrung sichtbar? Prüfe die exakte Reihenfolge der Zeilen in `common-auth` – PAM wertet sie sequenziell aus. **Musterlösung:** Nach Ablauf der 600 Sekunden oder nach `sudo faillock --user testuser --reset` ist das Konto wieder normal nutzbar.

7. Typische Fehler

Fehler	Ursache	Behebung
Niemand kann sich mehr anmelden	Fehlerhafte PAM-Konfiguration ohne offene Sicherheitssitzung	immer mit zweiter, offener Sitzung testen
`pam_faillock` sperrt root versehentlich dauerhaft	keine Ausnahme für administrative Konten konfiguriert	`root_unlock_time` separat setzen oder root-Konsolenzugriff sicherstellen
Passwort-Richtlinie wirkt nur bei neuen Passwörtern	bestehende, schwache Passwörter bleiben gültig	Passwortwechsel für Bestandskonten erzwingen (`chage -d 0`)

8. Zusammenfassung

> **Zusammenfassung**
>
> - PAM ist die zentrale, modulare Authentifizierungsschicht unter Linux.
> - `pam_pwquality` erzwingt Mindestanforderungen an neue Passwörter.
> - `pam_faillock` sperrt Konten nach einer konfigurierbaren Anzahl Fehlversuche.
> - Jede PAM-Änderung sollte mit einer zweiten, offenen Sitzung abgesichert getestet werden.

9. Kontrollfragen

1. Wofür steht die Abkürzung PAM? (leicht)
2. Was ist der Unterschied zwischen Authentifizierung und Autorisierung? (leicht)
3. Welchen Zweck erfüllt `pam_pwquality`? (mittel)
4. Warum sollte man PAM-Änderungen niemals ohne zweite, offene Sitzung testen? (mittel)
5. Eine zu niedrige `deny`-Schwelle bei `pam_faillock` schützt zwar gut vor Brute-Force, birgt aber ein eigenes Risiko. Welches, und wie lässt es sich abmildern? (schwer)

Lösungshinweise: 1. Pluggable Authentication Modules. 2. Authentifizierung prüft die Identität, Autorisierung prüft die Berechtigung für eine konkrete Aktion. 3. Es erzwingt Mindestanforderungen wie Länge und Zeichenvielfalt bei neu gesetzten Passwörtern. 4. Ein Konfigurationsfehler kann jede Anmeldung verhindern, auch die eigene. 5. Das Risiko eines absichtlich herbeigeführten Denial-of-Service gegen ein fremdes Konto; abmildern lässt sich das durch eine etwas höhere Schwelle oder zusätzliche, quellenbasierte Begrenzung wie Fail2ban (Band 2).

10. Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] den PAM-Ablauf anhand einer Konfigurationsdatei grob nachvollziehen kannst,
- [ ] eine Passwort-Richtlinie mit `pam_pwquality` erfolgreich eingerichtet hast,
- [ ] eine Kontosperrung mit `pam_faillock` ausgelöst und zurückgesetzt hast,
- [ ] erklären kannst, warum PAM-Änderungen besondere Vorsicht erfordern.

Kapitel 5: SSH-Sicherheit

Lernziele

- Du erklärst, warum Schlüsselauthentifizierung Passwörtern vorzuziehen ist.
- Du erzeugst ein Schlüsselpaar und richtest schlüsselbasierte Anmeldung ein.
- Du liest und passt zentrale Parameter der `sshd_config` begründet an.

1. Einführung

SSH ist der Standardweg, um Linux-Server fernzuwarten, und damit zugleich einer der am häufigsten aus dem Internet erreichbaren Dienste überhaupt. Die Grundlagen aus diesem Kapitel bereiten die tiefere Härtung in Band 2 vor.

2. Technische Grundlagen

OpenSSH sichert eine Verbindung über asymmetrische Kryptografie ab (Grundprinzip in Kapitel 10 vertieft): Ein **Schlüsselpaar** besteht aus einem privaten Schlüssel, der geheim auf dem Client bleibt, und einem öffentlichen Schlüssel, der auf dem Server hinterlegt wird. Zur Anmeldung beweist der Client, im Besitz des passenden privaten Schlüssels zu sein, ohne ihn jemals zu übertragen.

Beispiel 5.1: Der Ablauf einer schlüsselbasierten Anmeldung (Konzept)

```
1. Client sendet Verbindungswunsch an Server
2. Server prüft, ob einer der hinterlegten öffentlichen Schlüssel passt
3. Server sendet eine Challenge, die nur mit dem passenden privaten
   Schlüssel korrekt beantwortet werden kann
4. Client beantwortet die Challenge mit dem privaten Schlüssel
5. Server akzeptiert die Verbindung, ohne das Geheimnis selbst zu kennen
```

Jeder SSH-Server besitzt zusätzlich einen eigenen **Host-Key**, mit dem er sich gegenüber Clients ausweist. Beim ersten Verbindungsaufbau speichert der Client diesen Host-Key in `~/.ssh/known_hosts`; ändert er sich später unerwartet, warnt SSH deutlich, weil das auf einen möglichen Man-in-the-Middle-Angriff hindeuten kann.

3. Fachbegriffe

Der **SSH-Agent** hält entschlüsselte private Schlüssel für die Dauer einer Sitzung im Arbeitsspeicher vor, damit die Passphrase nicht bei jeder Verbindung erneut eingegeben werden muss. Die Datei `sshd_config` (serverseitig, meist unter `/etc/ssh/sshd_config`) konfiguriert das Verhalten des SSH-Servers; `ssh_config` (clientseitig) konfiguriert dagegen das Verhalten des SSH-Clients.

Passphrase bezeichnet das Passwort, mit dem ein privater Schlüssel selbst zusätzlich verschlüsselt wird – unabhängig vom Login-Passwort des Benutzerkontos. Ohne Passphrase schützt ein gestohlener privater Schlüssel überhaupt nicht mehr.

4. Praxis und Konfiguration

Beispiel 5.2: Ein Schlüsselpaar erzeugen (auf security-client)

```
$ ssh-keygen -t ed25519 -C "eugen@security-client"
```

`ed25519` gilt heute als empfohlenes, modernes Verfahren; das ältere RSA (`-t rsa -b 4096`) bleibt aus Kompatibilitätsgründen verbreitet, sollte bei neuen Schlüsseln aber nicht mehr die erste Wahl sein.

Beispiel 5.3: Öffentlichen Schlüssel auf den Server übertragen

```
$ ssh-copy-id benutzername@192.168.56.20
```

Beispiel 5.4: Zentrale sshd_config-Parameter

```
PermitRootLogin no
PasswordAuthentication no
PubkeyAuthentication yes
```

Parameter	Wirkung
`PermitRootLogin no`	direkter root-Login per SSH wird verhindert
`PasswordAuthentication no`	nur Schlüssel werden akzeptiert, keine Passwörter mehr
`PubkeyAuthentication yes`	Schlüsselauthentifizierung ist aktiviert (Standard)

Beispiel 5.5: Konfiguration übernehmen

```
$ sudo sshd -t     # Syntax der Konfiguration prüfen, bevor neu geladen wird
$ sudo systemctl reload sshd
```

WARNUNG – Sicherheitsrisiko

Setze `PasswordAuthentication no` niemals, bevor du nicht erfolgreich mit einem Schlüssel angemeldet warst. Prüfe das in einer zweiten, noch offenen Sitzung, bevor du die aktuelle schließt – sonst sperrst du dich möglicherweise selbst aus.

5. Sicherheitsbetrachtung

Passwort-basierte SSH-Anmeldung ist anfällig für Brute-Force-Versuche aus dem Internet, ein über Jahre hinweg beobachtbares, ständiges Hintergrundrauschen gegen jeden öffentlich erreichbaren SSH-Server. Schlüsselbasierte Anmeldung macht diese Angriffsklasse praktisch wirkungslos, weil kein Passwort existiert, das erraten werden könnte. Ein ungeschützter, unverschlüsselter privater Schlüssel auf einem gestohlenen Laptop stellt dagegen ein eigenständiges Risiko dar – deshalb gehört zu jedem Schlüssel eine Passphrase.

6. Praktische Laborübungen

Übung A: Schlüsselbasierte Anmeldung einrichten

Aufgabenstellung: Richte schlüsselbasierte Anmeldung zwischen `security-client` und `server1` ein und bestätige sie. **Ausgangssituation:** Bisher nur passwortbasierte Anmeldung möglich. **Voraussetzungen:** SSH-Zugriff mit Passwort funktioniert bereits. **Schritt-für-Schritt-Anleitung:** siehe Beispiel 5.2 und 5.3. **Erwartetes Ergebnis:** Eine neue SSH-Verbindung fragt nicht mehr nach einem Passwort, sondern höchstens nach der Passphrase des Schlüssels. **Überprüfung:** `ssh -v benutzername@192.168.56.20` zeigt in der ausführlichen Ausgabe „Authentication succeeded (publickey)“. **Fehleranalyse:** Weiterhin Passwortabfrage? Prüfe Rechte von `~/.ssh` (700) und `~/.ssh/authorized_keys` (600) auf dem Server – zu offene Rechte werden von SSH aus Sicherheitsgründen ignoriert. **Musterlösung:** Nach korrekter Einrichtung erfolgt die Anmeldung ohne Passwortabfrage, die Ausgabe von `ssh -v` bestätigt „publickey“ als erfolgreiche Methode.

Übung B: sshd_config gezielt härten und testen

Aufgabenstellung: Deaktiviere `PasswordAuthentication` erst nach erfolgreicher Übung A, und bestätige, dass root-Login per SSH nicht möglich ist. **Ausgangssituation:** Ergebnis aus Übung A. **Voraussetzungen:** Eine zweite, offene SSH-Sitzung als Sicherheitsnetz.
Schritt-für-Schritt-Anleitung: siehe Beispiel 5.4 und 5.5. **Erwartetes Ergebnis:** Ein Anmeldeversuch mit Passwort wird abgelehnt, ein Anmeldeversuch als `root` ebenfalls, unabhängig von der Methode. **Überprüfung:** `ssh root@192.168.56.20` liefert „Permission denied“, bevor überhaupt eine Authentifizierungsmethode abgefragt wird. **Fehleranalyse:** Änderung ohne Wirkung? Prüfe mit `sudo sshd -t`, ob die Konfigurationsdatei syntaktisch fehlerfrei ist, und ob `reload` tatsächlich erfolgreich war. **Musterlösung:** Nach korrekter Konfiguration akzeptiert `server1` ausschließlich schlüsselbasierte Anmeldungen regulärer, nicht-root-Konten.

7. Typische Fehler

Fehler	Ursache	Behebung
Aussperrung nach `PasswordAuthentication no`	Schlüssel vorher nicht getestet	immer erst Übung A, dann Übung B, mit zweiter Sitzung
Schlüssel wird trotz Einrichtung nicht akzeptiert	falsche Dateirechte auf `~/.ssh` oder `authorized_keys`	Rechte auf 700 bzw. 600 setzen
Privater Schlüssel ohne Passphrase erzeugt	Bequemlichkeit beim Erzeugen	Schlüssel mit Passphrase neu erzeugen, alten widerrufen

8. Zusammenfassung

Zusammenfassung

- Schlüsselbasierte Authentifizierung macht Passwort-Brute-Force gegen SSH wirkungslos.
- Host-Keys schützen vor unbemerktem Vertauschen des Servers.
- `PermitRootLogin no` und `PasswordAuthentication no` sind zentrale, aber erst nach erfolgreichem Schlüsseltest zu setzende Parameter.
- Ein privater Schlüssel ohne Passphrase ist bei Diebstahl ungeschützt.

9. Kontrollfragen

1. Woraus besteht ein SSH-Schlüsselpaar? (leicht)
2. Wozu dient die Datei `known_hosts`? (leicht)
3. Warum sollte `PasswordAuthentication no` erst nach erfolgreichem Schlüsseltest gesetzt werden? (mittel)
4. Welche Dateirechte erwartet SSH für `~/.ssh` und `authorized_keys`? (mittel)
5. Ein Kollege erzeugt einen SSH-Schlüssel ohne Passphrase, „weil das bequemer ist“. Welches konkrete Risiko entsteht dadurch, und wie würdest du es ihm erklären? (schwer)

Lösungshinweise: 1. Aus einem privaten Schlüssel (geheim) und einem öffentlichen Schlüssel (auf dem Server hinterlegt). 2. Sie speichert die Host-Keys bereits bekannter Server, um spätere Veränderungen zu erkennen. 3. Um sich nicht versehentlich selbst von jeder Anmeldemöglichkeit auszuschließen. 4. 700 für `~/.ssh`, 600 für `authorized_keys`. 5. Wird der private Schlüssel gestohlen (z. B. durch einen kompromittierten Laptop), kann er ohne jede weitere Hürde sofort verwendet werden; eine Passphrase fügt eine zusätzliche, vom Diebstahl unabhängige Schutzebene hinzu.

10. Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] ein SSH-Schlüsselpaar erzeugt und erfolgreich eingerichtet hast,
- [ ] erklären kannst, warum Host-Keys vor Man-in-the-Middle-Angriffen warnen,
- [ ] `sshd_config` gezielt und sicher angepasst hast, ohne dich auszusperren,
- [ ] den Unterschied zwischen Login-Passwort und Schlüssel-Passphrase erklären kannst.

Kapitel 6: Grundlagen der Netzwerksicherheit

Lernziele

- Du erklärst die für Sicherheit relevanten Grundlagen von TCP/IP.
- Du liest offene Ports und Netzwerkschnittstellen eines Systems aus.
- Du unterscheidest vertrauenswürdige von nicht vertrauenswürdigen Netzsegmenten.

1. Einführung

Jeder Netzwerkdienst, der auf einem System läuft, ist ein potenzieller Zugangspunkt. Bevor Kapitel 7 Firewalls behandelt, braucht es ein Grundverständnis dessen, was überhaupt gefiltert werden soll.

2. Technische Grundlagen

TCP/IP organisiert Kommunikation in Schichten; für die Netzwerksicherheit sind vor allem drei relevant: die **Vermittlungsschicht** (IP-Adressen, Routing), die **Transportschicht** (TCP/UDP, Ports) und die **Anwendungsschicht** (HTTP, SSH, DNS). Ein **Port** ist eine Nummer zwischen 0 und 65535, über die ein System mehrere gleichzeitig laufende Netzwerkdienste unterscheidet; Ports unter 1024 gelten traditionell als privilegiert und erfordern root-Rechte zum Binden (vergleiche Kapitel 2, Kontrollfrage 5).

Beispiel 6.1: Offene Ports und zugehörige Prozesse anzeigen

```
$ ss -tulpen
```

Spalte	Bedeutung
`-t`	TCP-Verbindungen
`-u`	UDP-Verbindungen
`-l`	nur lauschende (horchende) Sockets
`-p`	zugehöriger Prozess
`-e`	erweiterte Informationen (u. a. Benutzer)
`-n`	numerische Darstellung statt Namensauflösung

3. Fachbegriffe

Ein **vertrauenswürdiges Netzsegment** ist ein Netzbereich, dessen Teilnehmer und deren Absichten bekannt und kontrolliert sind, etwa das interne Serversegment einer Organisation. Ein **nicht vertrauenswürdiges Netzsegment**, meist das öffentliche Internet, enthält unbekannte, potenziell feindselige Teilnehmer. Diese Unterscheidung begründet, warum ein Server mit mehreren Netzwerkschnittstellen unterschiedliche Firewall-Regeln pro Schnittstelle braucht (Kapitel 7).

DNS (Domain Name System) löst menschenlesbare Namen in IP-Adressen auf. Da DNS-Antworten standardmäßig unauthentifiziert sind, kann eine gefälschte Antwort (DNS-Spoofing) einen Client auf ein falsches, bösartiges Ziel umleiten – ein Grund, warum sensible Verbindungen zusätzlich über

TLS-Zertifikate abgesichert werden (Kapitel 10).

4. Praxis und Konfiguration

Beispiel 6.2: Netzwerkschnittstellen und Routing anzeigen

```
$ ip addr show
$ ip route show
```

Beispiel 6.3: Eine Netzwerkschnittstelle über netplan/interfaces konfigurieren (Debian-Standard: /etc/network/interfaces)

```
auto eth0
iface eth0 inet static
    address 192.168.56.20/24
```

Beispiel 6.4: DNS-Auflösung testen

```
$ host debian.org
$ dig debian.org +short
```

5. Sicherheitsbetrachtung

> **WARNUNG – Sicherheitsrisiko**
>
> Ein Server mit zwei Netzwerkschnittstellen – einer im internen, vertrauenswürdigen Segment und einer im öffentlichen Internet – darf niemals dieselben, offenen Firewall-Regeln für beide Schnittstellen verwenden. Ein Dienst, der nur für das interne Segment gedacht ist, aber auch auf der öffentlichen Schnittstelle lauscht, ist ein klassisches, vermeidbares Einfallstor.

Jeder offene Port, der nicht aktiv gebraucht wird, vergrößert die Angriffsfläche ohne Gegenwert. Die in Übung A dieses Kapitels erstellte Übersicht bildet die Grundlage für die Firewall-Regeln in Kapitel 7: Nur, was wirklich gebraucht wird, sollte am Ende erlaubt sein.

6. Praktische Laborübungen

Übung A: Offene Ports vollständig dokumentieren

Aufgabenstellung: Erstelle eine vollständige, begründete Liste aller offenen Ports auf `server1`. **Ausgangssituation:** `server1`, Standardkonfiguration nach Kapitel 1-5. **Voraussetzungen:** SSH-Zugriff. **Schritt-für-Schritt-Anleitung:**

Befehle zu Übung A

```
$ ss -tulpen
$ ip addr show
```

Erwartetes Ergebnis: Eine Tabelle mit Port, Protokoll, zuständigem Prozess und Begründung, warum dieser Port offen ist. **Überprüfung:** Jeder gefundene Port hat einen Eintrag; kein Port bleibt unerklärt. **Fehleranalyse:** Ein Port lässt sich keinem bekannten Dienst zuordnen? Prüfe mit `systemctl list-units --type=service` nach passenden, aktiven Diensten. **Musterlösung:** Bei einem Minimalsystem nach Kapitel 1-5 sollte ausschließlich Port 22/tcp (SSH) erscheinen.

Übung B: Netzwerkschnittstelle und Routing nachvollziehen

Aufgabenstellung: Dokumentiere die Netzwerkkonfiguration von `server1` vollständig und prüfe die DNS-Auflösung. **Ausgangssituation:** wie Übung A. **Voraussetzungen:** keine zusätzlichen. **Schritt-für-Schritt-Anleitung:** siehe Beispiel 6.2 und 6.4. **Erwartetes Ergebnis:** Eine Übersicht aus IP-Adresse, Subnetzmaske, Standard-Gateway (sofern vorhanden) und erfolgreicher oder fehlgeschlagener DNS-Auflösung. **Überprüfung:** Da `server1` laut Laboraufbau keinen Internetzugang hat, sollte eine externe DNS-Abfrage fehlschlagen – ein erwartetes, korrektes Ergebnis, kein Fehler. **Fehleranalyse:** Funktioniert DNS dennoch? Prüfe, ob die VM versehentlich doch mit einem Bridged-Adapter statt dem internen Netzwerk konfiguriert wurde. **Musterlösung:** `ip addr show` bestätigt die statische Adresse aus dem Laboraufbau (Vorwort, Beispiel V.1); eine externe DNS-Abfrage schlägt erwartungsgemäß fehl.

7. Typische Fehler

Fehler	Ursache	Behebung
Unbegründeter, offener Port übersehen	Angriffsfläche nicht systematisch dokumentiert	regelmäßig `ss -tulpen` gegen die eigene Dokumentation abgleichen
Interne und öffentliche Schnittstelle gleich behandelt	fehlendes Bewusstsein für Netzsegmentierung	Firewall-Regeln immer pro Schnittstelle denken (Kapitel 7)
DNS-Antworten blind vertraut	Unkenntnis von DNS-Spoofing-Risiko	sensible Verbindungen zusätzlich über TLS-Zertifikatsprüfung absichern

8. Zusammenfassung

Zusammenfassung

- TCP/IP-Schichten, Ports und das Vertrauensmodell eines Netzsegments bilden die Grundlage jeder Firewall-Entscheidung.
- `ss -tulpen` macht offene Ports und zuständige Prozesse sichtbar.
- Interne und öffentliche Netzwerkschnittstellen brauchen unterschiedliche Regeln.
- DNS ist standardmäßig unauthentifiziert und daher kein alleiniger Vertrauensanker.

9. Kontrollfragen

1. Was ist ein Port, und in welchem Nummernbereich liegen privilegierte Ports? (leicht)
2. Welcher Befehl zeigt offene Ports samt zuständigem Prozess? (leicht)
3. Warum braucht ein Server mit zwei Netzwerkschnittstellen unterschiedliche Firewall-Regeln pro Schnittstelle? (mittel)
4. Was bedeutet es, dass DNS „standardmäßig unauthentifiziert“ ist? (mittel)
5. Ein Dienst soll nur innerhalb des internen Netzsegments erreichbar sein, lauscht aber derzeit auf allen Schnittstellen. Welche zwei grundsätzlich unterschiedlichen Lösungsansätze gibt es? (schwer)

Lösungshinweise: 1. Eine Nummer zwischen 0 und 65535 zur Unterscheidung von Diensten; privilegiert sind Ports unter 1024. 2. `ss -tulpen`. 3. Weil sonst ein nur intern gedachter Dienst auch aus dem nicht vertrauenswürdigen Segment erreichbar wäre. 4. Dass eine DNS-Antwort nicht kryptografisch bestätigt wird und sich deshalb fälschen lässt. 5. Entweder den Dienst selbst so

konfigurieren, dass er nur an die interne Adresse bindet, oder ihn weiterhin auf allen Schnittstellen laufen lassen, aber über eine Firewall-Regel (Kapitel 7) den Zugriff von der öffentlichen Schnittstelle aus zu blockieren.

10. Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] offene Ports eines Systems mit `ss -tulpen` vollständig auslesen kannst,
- [ ] Netzwerkschnittstellen und Routing mit `ip` anzeigen kannst,
- [ ] vertrauenswürdige von nicht vertrauenswürdigen Netzsegmenten unterscheiden kannst,
- [ ] erklären kannst, warum DNS-Antworten allein kein ausreichendes Vertrauen begründen.

Kapitel 7: Firewall-Grundlagen

Lernziele

- Du erklärst das Prinzip eines Paketfilters und den Unterschied zustandslos/zustandsbehaftet.
- Du richtest eine einfache Standard-Deny-Regel mit nftables ein.
- Du baust dieselbe Regel mit ufw nach und ordnest beide Ansätze ein.

1. Einführung

Nachdem Kapitel 6 sichtbar gemacht hat, welche Ports überhaupt offen sind, legt dieses Kapitel fest, welcher Netzwerkverkehr tatsächlich erlaubt sein soll. Die vollständige, produktionsreife Konfiguration folgt in Band 2; hier geht es um das Grundprinzip.

2. Technische Grundlagen

Ein **Paketfilter** entscheidet für jedes einzelne Netzwerkpaket anhand definierter Regeln, ob es durchgelassen, verworfen oder abgelehnt wird. Ein **zustandsloser** Filter betrachtet jedes Paket isoliert; ein **zustandsbehafteter** (Stateful) Filter erkennt dagegen, ob ein Paket zu einer bereits erlaubten, bestehenden Verbindung gehört, und muss die Rückrichtung einer Verbindung deshalb nicht gesondert freigeben.

Seit Debian 10 ist **nftables** das empfohlene, moderne Firewall-Framework im Kernel und löst das ältere `iptables` ab; Debian 13 nutzt nftables als Standard. **ufw** (Uncomplicated Firewall) ist eine vereinfachte, auf nftables aufbauende Oberfläche für den alltäglichen Gebrauch.

Beispiel 7.1: Grundstruktur einer nftables-Regelbasis (Konzept)

```
table inet filter {
    chain input {
        type filter hook input priority 0; policy drop;
        ct state established,related accept
        iif lo accept
        tcp dport 22 accept
    }
}
```

`policy drop` setzt die Grundregel auf „alles verwerfen, was nicht ausdrücklich erlaubt ist“ – das **Standard-Deny-Prinzip**, die sicherheitstechnisch empfohlene Grundhaltung gegenüber dem umgekehrten „alles erlauben, was nicht ausdrücklich verboten ist“.

3. Fachbegriffe

Eine **Chain** (Kette) ist eine geordnete Liste von Regeln, die für einen bestimmten Punkt im Paketfluss gilt, etwa `input` für eingehende Pakete. Eine **Policy** ist die Grundentscheidung einer Chain, falls keine ihrer Regeln zutrifft. `ct state established,related` nutzt die **Connection-Tracking**-Funktion des Kernels, um zustandsbehaftete Entscheidungen zu treffen.

4. Praxis und Konfiguration

Beispiel 7.2: nftables installieren und eine Grundregelbasis einrichten

```
$ sudo apt install nftables
$ sudo nano /etc/nftables.conf
```

Beispiel 7.3: Inhalt von /etc/nftables.conf

```
#!/usr/sbin/nft -f
flush ruleset

table inet filter {
    chain input {
        type filter hook input priority 0; policy drop;
        ct state established,related accept
        iif lo accept
        tcp dport 22 accept
        ct state invalid drop
    }
    chain forward {
        type filter hook forward priority 0; policy drop;
    }
    chain output {
        type filter hook output priority 0; policy accept;
    }
}
```

Beispiel 7.4: Regeln aktivieren

```
$ sudo nft -c -f /etc/nftables.conf   # Syntax prüfen (-c = check only)
$ sudo systemctl enable --now nftables
```

Beispiel 7.5: Dieselbe Regel mit ufw

```
$ sudo apt install ufw
$ sudo ufw default deny incoming
$ sudo ufw default allow outgoing
$ sudo ufw allow 22/tcp
$ sudo ufw enable
```

5. Sicherheitsbetrachtung

> **WARNUNG – Sicherheitsrisiko**
>
> Aktiviere eine neue, restriktive Firewall-Regel niemals, ohne vorher sicherzustellen, dass die Regel für SSH (Port 22) bereits enthalten ist. Eine Standard-Deny-Policy ohne SSH-Ausnahme trennt eine Fernverbindung sofort und unwiderruflich, bis jemand lokal Zugriff auf die Konsole der VM hat.

`nftables` bietet die größere Kontrolle und wird in Band 2 für komplexere Szenarien vertieft; `ufw` eignet sich für einfache, schnell nachvollziehbare Regelwerke und ist für den Einstieg weniger fehleranfällig. Beide erzeugen am Ende dieselbe zugrunde liegende Kernel-Filterung.

6. Praktische Laborübungen

Übung A: Standard-Deny mit nftables umsetzen

Aufgabenstellung: Richte die Regelbasis aus Beispiel 7.2-7.4 ein und bestätige, dass nur SSH erreichbar bleibt. **Ausgangssituation:** `server1`, bisher keine aktive Firewall. **Voraussetzungen:** sudo-Rechte, zweite offene SSH-Sitzung als Sicherheitsnetz. **Schritt-für-Schritt-Anleitung:** siehe Beispiel 7.2-7.4. **Erwartetes Ergebnis:** `nft list ruleset` zeigt die aktive Konfiguration; eine neue SSH-Verbindung funktioniert weiterhin, ein testweiser Verbindungsversuch auf einen anderen, geschlossenen Port schlägt fehl. **Überprüfung:** `sudo nft -c -f /etc/nftables.conf` meldet keinen Syntaxfehler, bevor die Regeln aktiv geschaltet werden. **Fehleranalyse:** Verbindung verloren? Über die Konsole der Virtualisierungssoftware (nicht per SSH) anmelden und `sudo systemctl stop nftables` ausführen, um den Normalzustand wiederherzustellen. **Musterlösung:** Nach korrekter Einrichtung bleibt ausschließlich Port 22/tcp erreichbar, alle anderen eingehenden Verbindungsversuche werden verworfen.

Übung B: Dieselbe Regel mit ufw nachbauen

Aufgabenstellung: Deaktiviere die nftables-Regeln aus Übung A testweise und baue dieselbe Grundregel mit ufw nach. **Ausgangssituation:** Ergebnis aus Übung A. **Voraussetzungen:** wie Übung A. **Schritt-für-Schritt-Anleitung:**

Befehle zu Übung B

```
$ sudo systemctl stop nftables
$ sudo systemctl disable nftables
# dann Beispiel 7.5 ausführen
$ sudo ufw status verbose
```

Erwartetes Ergebnis: `ufw status verbose` zeigt „Status: active“, eingehend „deny (incoming)“ mit einer Ausnahme für 22/tcp. **Überprüfung:** Dasselbe Verhalten wie in Übung A: SSH funktioniert, andere Ports sind blockiert. **Fehleranalyse:** `ufw` meldet einen Konflikt mit nftables? Stelle sicher, dass der nftables-Dienst tatsächlich gestoppt und deaktiviert wurde, bevor `ufw` aktiviert wird. **Musterlösung:** `ufw status verbose` zeigt eine kurze, klar lesbare Regelliste mit demselben Ergebnis wie die deutlich ausführlichere nftables-Konfiguration aus Übung A.

7. Typische Fehler

Fehler	Ursache	Behebung
Aussperrung nach Aktivierung der Firewall	SSH-Regel fehlt oder steht an falscher Position	Regeln vor Aktivierung mit `-c`/`--dry-run` prüfen, SSH-Regel immer zuerst
nftables und ufw gleichzeitig aktiv	beide verwalten dieselbe Kernel-Filterung, Konflikte entstehen	immer nur eines der beiden Systeme aktiv betreiben
Policy „accept“ statt „drop“ gesetzt	Verwechslung der beiden Grundhaltungen	Standard-Deny bewusst als Ausgangspunkt wählen

8. Zusammenfassung

Zusammenfassung

- Ein Paketfilter entscheidet pro Paket über Durchlass, Verwurf oder Ablehnung; zustandsbehaftete Filter erkennen bestehende Verbindungen automatisch.
- nftables ist das moderne, im Kernel integrierte Firewall-Framework unter Debian 13.
- ufw vereinfacht die Bedienung für alltägliche Szenarien.
- Standard-Deny mit expliziter SSH-Ausnahme ist die sichere Grundhaltung für jeden Server.

9. Kontrollfragen

1. Was unterscheidet einen zustandslosen von einem zustandsbehafteten Paketfilter? (leicht)
2. Welches Firewall-Framework ist unter Debian 13 Standard? (leicht)
3. Was bedeutet das Standard-Deny-Prinzip? (mittel)
4. Warum sollte die SSH-Regel vor jeder restriktiven Policy-Änderung bereits aktiv sein? (mittel)
5. Nenne einen Grund, aus Verwaltungssicht trotz vorhandenem ufw gezielt mit nftables direkt zu arbeiten. (schwer)

Lösungshinweise: 1. Zustandsbehaftet erkennt bestehende Verbindungen automatisch, zustandslos bewertet jedes Paket isoliert. 2. nftables. 3. Alles verwerfen, was nicht ausdrücklich erlaubt ist. 4. Weil sonst eine aktivierte Deny-Policy die eigene Fernverbindung sofort trennen würde. 5. Komplexere Regelwerke (z. B. mit mehreren Chains, Sets oder differenzierten Protokollierungsregeln, Band 2) lassen sich mit nftables direkt feiner steuern, als es die vereinfachte ufw-Oberfläche erlaubt.

10. Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] den Unterschied zwischen zustandslosem und zustandsbehaftetem Filtern erklären kannst,
- [ ] eine funktionierende Standard-Deny-Regel mit nftables eingerichtet hast,
- [ ] dieselbe Regel mit ufw nachgebaut hast,
- [ ] erklären kannst, warum die SSH-Ausnahme vor jeder restriktiven Policy stehen muss.

Kapitel 8: Sicherheitsupdates und Paketverwaltung

Lernziele

- Du erklärst, wie APT Pakete signiert und vertrauenswürdig bezieht.
- Du richtest automatisierte Sicherheitsupdates mit unattended-upgrades ein.
- Du ordnest ein Sicherheitsadvisory einem installierten Paket zu.

1. Einführung

Die sorgfältigste Konfiguration verliert ihren Wert, wenn bekannt gewordene Schwachstellen nicht zeitnah geschlossen werden. Dieses Kapitel verbindet die CVE-Recherche aus Kapitel 1 mit der praktischen Umsetzung: Updates zuverlässig einspielen, ohne dabei blind jeder Quelle zu vertrauen.

2. Technische Grundlagen

APT (Advanced Package Tool) verwaltet Pakete unter Debian über konfigurierte Quellen (Repositories), deren Inhalte kryptografisch signiert sind. Jedes Repository wird durch einen öffentlichen Schlüssel im lokalen **Keyring** vertrauenswürdig gemacht; APT installiert nur Pakete, deren Signatur zu einem bekannten, vertrauenswürdigen Schlüssel passt.

Beispiel 8.1: Grundlegende APT-Befehle

```
$ sudo apt update        # Paketlisten der konfigurierten Quellen aktualisieren
$ sudo apt list --upgradable
$ sudo apt full-upgrade  # Pakete inkl. Abhängigkeitsänderungen aktualisieren
```

Debian veröffentlicht sicherheitsrelevante Updates für die Stable-Distribution zusätzlich über ein eigenes **Security-Repository**, getrennt vom regulären Stable-Zweig, damit Sicherheitskorrekturen schneller verteilt werden können als reguläre Funktionsupdates.

3. Fachbegriffe

Ein **DSA** (Debian Security Advisory) ist die offizielle Ankündigung einer behobenen, sicherheitsrelevanten Schwachstelle in einem Debian-Paket, veröffentlicht über die Debian-Sicherheitsinfrastruktur. **no-dsa** im Debian Security Tracker bedeutet, dass ein CVE zwar bekannt, aber als nicht schwerwiegend genug für ein eigenes Advisory eingestuft wurde – es kann trotzdem über ein reguläres Update behoben sein. **End-of-Life** (EOL) bezeichnet den Zeitpunkt, ab dem ein Softwarezweig keine Sicherheitsupdates mehr erhält.

4. Praxis und Konfiguration

Beispiel 8.2: unattended-upgrades installieren

```
$ sudo apt install unattended-upgrades apt-listchanges
$ sudo dpkg-reconfigure --priority=low unattended-upgrades
```

Beispiel 8.3: Zentrale Konfiguration (Auszug aus /etc/apt/apt.conf.d/50unattended-upgrades)

```
Unattended-Upgrade::Origins-Pattern {
    "origin=Debian,codename=${distro_codename}-security";
};
```

Diese Zeile beschränkt automatische Updates gezielt auf das Security-Repository – reguläre, funktionsändernde Updates bleiben eine bewusste, manuelle Entscheidung.

Beispiel 8.4: Trockenlauf der automatischen Aktualisierung

```
$ sudo unattended-upgrade --dry-run --debug
```

5. Sicherheitsbetrachtung

> **WARNUNG – Sicherheitsrisiko**
>
> Automatische Updates ohne jede Kontrolle bergen ein eigenes Risiko: Eine fehlerhafte Paketaktualisierung kann einen produktiven Dienst stören. Die hier gezeigte Beschränkung auf das Security-Repository reduziert dieses Risiko, ersetzt aber nicht die regelmäßige, manuelle Prüfung größerer Funktionsupdates.

Ein installiertes, aber über Jahre nicht aktualisiertes Paket mit öffentlich bekannten Schwachstellen ist eines der am leichtesten vermeidbaren Sicherheitsrisiken überhaupt – und zugleich eines der häufigsten in der Praxis, weil Updates im laufenden Betrieb leicht aus dem Blick geraten.

6. Praktische Laborübungen

Übung A: Automatische Sicherheitsupdates einrichten

Aufgabenstellung: Richte `unattended-upgrades` nach Beispiel 8.2/8.3 ein und bestätige die Konfiguration mit einem Trockenlauf. **Ausgangssituation:** `server1`, noch keine automatisierten Updates. **Voraussetzungen:** sudo-Rechte. **Schritt-für-Schritt-Anleitung:** siehe Beispiel 8.2-8.4. **Erwartetes Ergebnis:** Der Trockenlauf listet auf, welche Pakete im Security-Repository aktuell ein Update erhalten würden, ohne es tatsächlich einzuspielen. **Überprüfung:** `cat /etc/apt/apt.conf.d/50unattended-upgrades` zeigt die Origins-Pattern-Zeile aus Beispiel 8.3. **Fehleranalyse:** Trockenlauf zeigt nichts an? Das ist normal, wenn `server1` bereits vollständig aktuell ist – prüfe testweise mit `sudo apt update && sudo apt list --upgradable`. **Musterlösung:** Eine vollständige Lösung zeigt die Konfigurationsdatei, den Trockenlauf-Befehl und seine Ausgabe, mit kurzer Einordnung, ob und warum Pakete betroffen wären.

Übung B: Ein Sicherheitsadvisory einem Paket zuordnen

Aufgabenstellung: Finde für ein auf `server1` installiertes Paket ein reales DSA oder CVE und ordne es konkret zu. **Ausgangssituation:** Paketliste aus Kapitel 1, Übung B, liegt vor. **Voraussetzungen:** Internetzugang von `security-client`. **Schritt-für-Schritt-Anleitung:**

Vorgehen zu Übung B

```
1. Installierte Version eines Pakets ermitteln: $ dpkg -l | grep openssh-server
2. security-tracker.debian.org nach diesem Paket durchsuchen
3. Ein Advisory oder CVE mit Status "resolved"/"fixed" oder "open" notieren
4. Prüfen, ob die installierte Version laut Tracker bereits den Fix enthält
```

Erwartetes Ergebnis: Eine klare Aussage, ob `server1` für das gefundene CVE aktuell verwundbar ist oder nicht. **Überprüfung:** Die installierte Versionsnummer aus `dpkg -l` lässt sich mit der im Tracker genannten „fixed version“ vergleichen. **Fehleranalyse:** Unklar, welche Version welche enthält? Debian nutzt eigene Versionsnummern-Suffixe; der Security Tracker selbst zeigt in der Regel direkt, ab welcher Version ein Fix vorliegt. **Musterlösung:** Eine vollständige Lösung nennt Paket, installierte Version, gefundenes CVE/DSA und eine begründete Aussage zur Verwundbarkeit.

7. Typische Fehler

Fehler	Ursache	Behebung
Updates „irgendwann mal von Hand“ eingespielt	kein automatisierter Prozess vorhanden	`unattended-upgrades` für das Security-Repository einrichten
Jedes Update ungeprüft als sicher angenommen	fehlendes Verständnis für Repository-Herkunft	nur konfigurierte, signierte Quellen verwenden, keine manuell hinzugefügten, ungeprüften Quellen
„no-dsa“ mit „ungefährlich“ verwechselt	Missverständnis der Tracker-Terminologie	„no-dsa“ bedeutet nur kein eigenes Advisory, nicht zwingend harmlos

8. Zusammenfassung

Zusammenfassung

- APT installiert nur signierte Pakete aus vertrauenswürdigen, konfigurierten Quellen.
- Das Debian-Security-Repository verteilt sicherheitsrelevante Updates getrennt vom regulären Stable-Zweig.
- `unattended-upgrades`, auf das Security-Repository beschränkt, automatisiert sicherheitsrelevante Updates risikobewusst.
- Der Debian Security Tracker erlaubt, ein installiertes Paket konkret gegen bekannte CVEs zu prüfen.

9. Kontrollfragen

1. Wofür steht APT? (leicht)
2. Was ist ein DSA? (leicht)
3. Warum verteilt Debian Sicherheitsupdates über ein eigenes Repository? (mittel)
4. Was bedeutet „no-dsa“ im Debian Security Tracker? (mittel)
5. Ein Server soll möglichst stabil laufen, aber zeitnah sicherheitsrelevante Updates erhalten. Wie lässt sich das mit `unattended-upgrades` konkret umsetzen, ohne gleichzeitig reguläre Funktionsupdates automatisch einzuspielen? (schwer)

Lösungshinweise: 1. Advanced Package Tool. 2. Debian Security Advisory, die offizielle Ankündigung einer behobenen Sicherheitslücke. 3. Damit sicherheitsrelevante Korrekturen unabhängig vom regulären Veröffentlichungszyklus schneller verteilt werden können. 4. Dass ein CVE zwar erfasst, aber nicht als eigenständiges Advisory veröffentlicht wurde. 5. Durch Beschränkung der `Origins-Pattern`-Konfiguration auf `${distro_codename}-security`, wie in Beispiel 8.3 gezeigt, sodass nur das Security-Repository automatisiert eingespielt wird.

10. Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] erklären kannst, wie APT Paketsignaturen prüft,
- [ ] `unattended-upgrades` auf das Security-Repository beschränkt eingerichtet hast,
- [ ] ein reales CVE oder DSA im Debian Security Tracker einem installierten Paket zugeordnet hast,
- [ ] den Unterschied zwischen „no-dsa“ und „harmlos“ erklären kannst.

Kapitel 9: Linux-Protokollierung und Systemüberwachung

Lernziele

- Du unterscheidest journald und rsyslog und ihre jeweilige Rolle.
- Du durchsuchst Protokolle gezielt mit journalctl nach sicherheitsrelevanten Ereignissen.
- Du richtest eine einfache Log-Rotation ein.

1. Einführung

Ohne Protokollierung bleibt jede Sicherheitsmaßnahme aus den vorherigen Kapiteln unüberprüfbar: Hat die Firewall-Regel tatsächlich etwas blockiert? Hat jemand versucht, sich mit einem falschen Passwort anzumelden? Dieses Kapitel legt die Grundlage, auf der die vertiefte Analyse in Band 3 aufbaut.

2. Technische Grundlagen

journald, Teil von systemd, sammelt strukturierte Protokolldaten von Kernel, Diensten und Anwendungen zentral und binär, durchsuchbar über `journalctl`. Das klassische **rsyslog** verarbeitet textbasierte Protokolle nach dem traditionellen Syslog-Standard und schreibt sie in Dateien wie `/var/log/syslog` oder `/var/log/auth.log`. Unter Debian 13 arbeiten beide Systeme meist parallel: journald sammelt zentral, rsyslog kann journald-Daten zusätzlich in klassische Textdateien weiterschreiben.

Beispiel 9.1: Protokolle mit journalctl durchsuchen

```
$ journalctl -u ssh.service --since "1 hour ago"
$ journalctl -p err      # nur Fehler-Priorität und höher
```

3. Fachbegriffe

Log-Rotation bezeichnet den automatisierten Prozess, Protokolldateien nach einer bestimmten Größe oder Zeitspanne zu archivieren, zu komprimieren und alte Stände zu löschen – ohne Rotation würden Logs unbegrenzt wachsen. `/var/log/auth.log` protokolliert Authentifizierungsereignisse (Anmeldungen, `sudo`-Aufrufe), `/var/log/syslog` allgemeine Systemereignisse.

4. Praxis und Konfiguration

Beispiel 9.2: Fehlgeschlagene Anmeldeversuche finden

```
$ grep "Failed password" /var/log/auth.log
$ journalctl -u ssh.service | grep "Failed password"
```

Beispiel 9.3: logrotate für ein eigenes Anwendungslog einrichten

```
$ sudo nano /etc/logrotate.d/meineanwendung
```

Beispiel 9.4: Inhalt der logrotate-Konfiguration

```
/var/log/meineanwendung/*.log {
    weekly
    rotate 4
    compress
    missingok
    notifempty
}
```

Parameter	Bedeutung
`weekly`	Rotation einmal pro Woche
`rotate 4`	vier archivierte Stände aufbewahren
`compress`	archivierte Stände komprimieren
`missingok`	kein Fehler, wenn die Logdatei fehlt
`notifempty`	keine Rotation, wenn die Datei leer ist

5. Sicherheitsbetrachtung

> **WARNUNG – Sicherheitsrisiko**
>
> Ein Protokoll, das existiert, aber nie ausgewertet wird, schützt niemanden. Logging ist die Voraussetzung für Erkennung, nicht die Erkennung selbst – diese Unterscheidung wird in Band 3 bei Intrusion Detection und SIEM-Systemen vertieft.

Protokolldateien selbst sind ein Angriffsziel: Wer Schreibzugriff auf `/var/log/auth.log` erlangt, kann Spuren eines Angriffs nachträglich verändern oder löschen. Die Dateirechte dieses Verzeichnisses (Kapitel 3) sind deshalb ebenso sicherheitsrelevant wie der Inhalt selbst.

6. Praktische Laborübungen

Übung A: Fehlgeschlagene Anmeldeversuche gezielt auswerten

Aufgabenstellung: Erzeuge absichtlich einige fehlgeschlagene SSH-Anmeldeversuche und werte sie aus. **Ausgangssituation:** `server1` mit aktivem SSH-Dienst. **Voraussetzungen:** Zugriff auf `security-client`. **Schritt-für-Schritt-Anleitung:**

Befehle zu Übung A

```
# Von security-client: mehrfach mit absichtlich falschem Passwort versuchen
$ ssh testuser@192.168.56.20
# Danach auf server1:
$ grep "Failed password" /var/log/auth.log
$ journalctl -u ssh.service --since "10 minutes ago"
```

Erwartetes Ergebnis: Jeder Fehlversuch erscheint als eigener Eintrag mit Zeitstempel, Benutzername und Quelladresse. **Überprüfung:** Die Anzahl der gefundenen Einträge stimmt mit der Anzahl deiner tatsächlichen Fehlversuche überein. **Fehleranalyse:** Keine Einträge? Prüfe, ob rsyslog aktiv ist (`systemctl status rsyslog`) oder weiche vollständig auf `journalctl` aus. **Musterlösung:** Eine vollständige Lösung zeigt die exakten Log-Zeilen und benennt Zeitstempel, Quelladresse und betroffenes Konto für jeden Versuch.

Übung B: Log-Rotation für ein eigenes Log einrichten

Aufgabenstellung: Lege ein Testverzeichnis mit einer Beispiel-Logdatei an und richte Rotation nach Beispiel 9.3/9.4 ein. **Ausgangssituation:** Kein eigenes Anwendungslog vorhanden. **Voraussetzungen:** sudo-Rechte. **Schritt-für-Schritt-Anleitung:**

Befehle zu Übung B

```
$ sudo mkdir -p /var/log/meineanwendung
$ sudo touch /var/log/meineanwendung/test.log
$ echo "Testeintrag" | sudo tee -a /var/log/meineanwendung/test.log
# logrotate-Konfiguration aus Beispiel 9.3/9.4 anlegen
$ sudo logrotate --debug /etc/logrotate.d/meineanwendung
```

Erwartetes Ergebnis: `logrotate --debug` zeigt, dass die Konfiguration erkannt wird und (abhängig vom Zeitpunkt) eine Rotation anstehen würde oder nicht. **Überprüfung:** `logrotate --force /etc/logrotate.d/meineanwendung` erzeugt tatsächlich eine komprimierte, archivierte Logdatei. **Fehleranalyse:** Fehler bei der Ausführung? Prüfe die Dateirechte des Log-Verzeichnisses und die Syntax der Konfigurationsdatei. **Musterlösung:** Nach `--force` liegt eine Datei wie `test.log.1.gz` im Verzeichnis, das Original ist geleert oder neu angelegt.

7. Typische Fehler

Fehler	Ursache	Behebung
Logs vorhanden, aber nie geprüft	kein Prozess zur regelmäßigen Auswertung	mindestens eine Routine zur stichprobenartigen Prüfung einführen
Logdateien wachsen unbegrenzt	fehlende Log-Rotation	`logrotate` für jedes eigene Anwendungslog konfigurieren
Protokolldateien für alle Benutzer lesbar	Dateirechte nicht geprüft	Zugriff auf `/var/log` gezielt auf berechtigte Konten beschränken

8. Zusammenfassung

Zusammenfassung

- journald sammelt zentral und strukturiert, rsyslog schreibt klassische Textdateien.
- `journalctl` und `grep` erlauben gezielte Suche nach sicherheitsrelevanten Ereignissen.
- Ohne Log-Rotation wachsen Protokolldateien unbegrenzt.
- Logging allein schützt nicht; erst die Auswertung macht daraus einen Sicherheitsgewinn.

9. Kontrollfragen

1. Was ist der Unterschied zwischen journald und rsyslog? (leicht)
2. Welcher Befehl durchsucht die systemd-Protokolle gezielt? (leicht)
3. Warum reicht vorhandenes Logging allein nicht als Schutzmaßnahme? (mittel)
4. Wozu dient Log-Rotation? (mittel)
5. Welche Dateirechte sollten für `/var/log/auth.log` gelten, und warum ist das aus Sicht eines Angreifers relevant? (schwer)

Lösungshinweise: 1. journald sammelt strukturiert und zentral, rsyslog schreibt klassische Textprotokolle. 2. `journalctl`. 3. Weil ein Protokoll, das nie geprüft wird, einen laufenden Angriff nicht verhindert. 4. Sie begrenzt das Wachstum von Protokolldateien durch Archivierung, Komprimierung und Löschung alter Stände. 5. Nur root und berechtigte Systemdienste sollten schreiben dürfen; ein Angreifer mit Schreibzugriff könnte sonst eigene Spuren nachträglich verändern oder löschen.

10. Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] mit `journalctl` gezielt nach einem Dienst und Zeitraum filtern kannst,
- [ ] fehlgeschlagene Anmeldeversuche in `/var/log/auth.log` findest,
- [ ] eine funktionierende Log-Rotation für ein eigenes Log eingerichtet hast,
- [ ] erklären kannst, warum Protokollrechte selbst sicherheitsrelevant sind.

Kapitel 10: Grundlagen der Verschlüsselung

Lernziele

- Du unterscheidest symmetrische von asymmetrischer Verschlüsselung und beide von Hashing.
- Du legst eine mit LUKS verschlüsselte Partition an und bindest sie ein.
- Du verschlüsselst und entschlüsselst eine Datei mit GPG.

1. Einführung

Dieses letzte Kapitel von Band 1 schließt den Kreis: SSH aus Kapitel 5 nutzt asymmetrische Kryptografie, Passwörter aus Kapitel 4 werden gehasht statt verschlüsselt, und ein gestohlener Datenträger bleibt ohne Verschlüsselung trotz aller anderen Maßnahmen lesbar.

2. Technische Grundlagen

Symmetrische Verschlüsselung nutzt denselben Schlüssel zum Ver- und Entschlüsseln; sie ist schnell, erfordert aber einen sicheren Weg, den Schlüssel zwischen beiden Seiten zu teilen. **Asymmetrische Verschlüsselung** nutzt ein Schlüsselpaar (öffentlich/privat, vergleiche Kapitel 5): Was mit dem öffentlichen Schlüssel verschlüsselt wird, lässt sich nur mit dem passenden privaten Schlüssel entschlüsseln. In der Praxis kombiniert TLS (die Grundlage von HTTPS) beide Verfahren: asymmetrisch für den anfänglichen Schlüsselaustausch, symmetrisch für die anschließende, schnellere Datenübertragung.

Beispiel 10.1: TLS-Grundprinzip (stark vereinfacht)

```
1. Client und Server einigen sich asymmetrisch auf einen gemeinsamen,
   geheimen Sitzungsschlüssel
2. Die eigentlichen Daten werden anschließend symmetrisch mit diesem
   Sitzungsschlüssel verschlüsselt übertragen
```

WARNUNG – Sicherheitsrisiko

Hashing ist keine Verschlüsselung. Ein Hash lässt sich nicht wieder in die ursprünglichen Daten zurückverwandeln, eine Verschlüsselung dagegen schon, mit dem passenden Schlüssel. Passwörter werden deshalb gehasht (Kapitel 4), Dateien oder Datenträger dagegen verschlüsselt. Diese beiden Begriffe werden in der Praxis häufig verwechselt.

3. Fachbegriffe

LUKS (Linux Unified Key Setup) ist der Standard für vollständige Festplatten- bzw. Partitionsverschlüsselung unter Linux; `cryptsetup` ist das zugehörige Kommandozeilenwerkzeug. **GPG** (GNU Privacy Guard) implementiert asymmetrische Verschlüsselung und digitale Signaturen für einzelne Dateien und E-Mails. Eine **digitale Signatur** bestätigt mit dem privaten Schlüssel des Absenders, dass eine Nachricht tatsächlich von ihm stammt und seit der Signatur unverändert ist – ein Beitrag zur Integrität (Kapitel 1), nicht zur Vertraulichkeit.

4. Praxis und Konfiguration

Beispiel 10.2: Eine LUKS-verschlüsselte Partition anlegen (Testpartition oder Datei-Image)

```
$ sudo cryptsetup luksFormat /dev/sdb1
$ sudo cryptsetup luksOpen /dev/sdb1 testcrypt
$ sudo mkfs.ext4 /dev/mapper/testcrypt
$ sudo mount /dev/mapper/testcrypt /mnt/testcrypt
```

Beispiel 10.3: Die Partition wieder aushängen und schließen

```
$ sudo umount /mnt/testcrypt
$ sudo cryptsetup luksClose testcrypt
```

Beispiel 10.4: Ein GPG-Schlüsselpaar erzeugen und eine Datei verschlüsseln

```
$ gpg --full-generate-key
$ gpg --encrypt --recipient "eugen@example.org" geheim.txt
$ gpg --decrypt geheim.txt.gpg > geheim_entschluesselt.txt
```

5. Sicherheitsbetrachtung

> **WARNUNG – Sicherheitsrisiko**
>
> `cryptsetup luksFormat` überschreibt den gewählten Datenträger unwiderruflich. Führe diesen Befehl ausschließlich gegen eine eigens dafür vorgesehene Testpartition oder ein Datei-Image aus, niemals gegen eine Partition mit vorhandenen, noch benötigten Daten.

Eine verschlüsselte Partition schützt Daten nur, solange sie ausgehängt (geschlossen) ist – bei einer eingebundenen, entschlüsselten Partition hat jeder mit Systemzugriff dieselben Rechte wie bei einer unverschlüsselten Platte. LUKS schützt also primär gegen den physischen Diebstahl eines ausgeschalteten Datenträgers, nicht gegen einen Angriff auf ein laufendes, bereits entschlüsseltes System.

6. Praktische Laborübungen

Übung A: Eine Datei mit GPG verschlüsseln und entschlüsseln

Aufgabenstellung: Erzeuge ein GPG-Schlüsselpaar, verschlüssele eine Testdatei und entschlüssele sie wieder. **Ausgangssituation:** Kein GPG-Schlüssel vorhanden. **Voraussetzungen:** `gpg` installiert (bei Debian und Kali standardmäßig vorhanden). **Schritt-für-Schritt-Anleitung:** siehe Beispiel 10.4. **Erwartetes Ergebnis:** Die entschlüsselte Datei ist inhaltlich identisch mit dem Original. **Überprüfung:** `diff geheim.txt geheim_entschluesselt.txt` liefert keine Ausgabe (keine Unterschiede). **Fehleranalyse:** Entschlüsselung schlägt fehl? Prüfe mit `gpg --list-keys`, ob der passende private Schlüssel tatsächlich im lokalen Schlüsselbund vorhanden ist. **Musterlösung:** `diff` ohne Ausgabe bestätigt, dass Ver- und Entschlüsselung verlustfrei funktioniert haben.

Übung B: Eine LUKS-verschlüsselte Testpartition anlegen

Aufgabenstellung: Lege nach Beispiel 10.2 eine verschlüsselte Testpartition oder ein Datei-Image an, binde sie ein, lege eine Testdatei darauf ab, und hänge sie wieder aus. **Ausgangssituation:** Eine zusätzliche, leere virtuelle Festplatte oder ein Datei-Image ohne vorhandene Daten. **Voraussetzungen:** `cryptsetup` installiert, root-Rechte. **Schritt-für-Schritt-Anleitung:** siehe Beispiel 10.2 und 10.3; alternativ statt einer echten Partition ein Datei-Image mit `dd if=/dev/zero of=testimage.img bs=1M count=100` und `losetup` anlegen. **Erwartetes Ergebnis:** Nach `luksClose` ist der Inhalt ohne erneutes `luksOpen` samt Passphrase nicht lesbar. **Überprüfung:** Ein Versuch, das Dateisystem direkt auf dem unentschlüsselten Blockgerät einzubinden, schlägt fehl oder zeigt keinen lesbaren Inhalt. **Fehleranalyse:** `luksFormat` schlägt fehl? Prüfe, ob das gewählte Gerät tatsächlich existiert und nicht bereits eingebunden ist. **Musterlösung:** Nach `luksOpen` mit korrekter Passphrase ist die Partition normal nutzbar; nach `luksClose` ist sie ohne erneute Authentifizierung nicht lesbar.

7. Typische Fehler

Fehler	Ursache	Behebung
Hashing und Verschlüsselung verwechselt	Begriffe klingen ähnlich, sind aber grundverschieden	Passwörter werden gehasht (nicht umkehrbar), Daten werden verschlüsselt (umkehrbar)
`luksFormat` gegen die falsche Partition ausgeführt	Gerätename nicht vorher geprüft	vor jedem `luksFormat` mit `lsblk` die Zuordnung bestätigen
Verschlüsselte Partition als dauerhaft geschützt angenommen	Schutz gilt nur im ausgehängten Zustand	zusätzliche Maßnahmen für ein laufendes, entschlüsseltes System einplanen

8. Zusammenfassung

Zusammenfassung

- Symmetrische Verschlüsselung nutzt einen gemeinsamen, asymmetrische ein Schlüsselpaar.
- Hashing ist nicht umkehrbar und damit grundlegend verschieden von Verschlüsselung.
- LUKS schützt Datenträger im ausgehängten Zustand, GPG schützt einzelne Dateien und Nachrichten.
- Verschlüsselung schützt nicht gegen Zugriff auf ein laufendes, bereits entschlüsseltes System.

9. Kontrollfragen

1. Was unterscheidet symmetrische von asymmetrischer Verschlüsselung? (leicht)
2. Warum ist Hashing keine Verschlüsselung? (leicht)
3. Wofür steht LUKS, und wofür wird es eingesetzt? (mittel)
4. Wie kombiniert TLS symmetrische und asymmetrische Verfahren? (mittel)
5. Ein Laptop mit LUKS-verschlüsselter Festplatte wird im eingeschalteten, entsperrten Zustand gestohlen. Schützt die Verschlüsselung in diesem Fall die Daten? Begründe. (schwer)

Lösungshinweise: 1. Symmetrisch nutzt denselben Schlüssel für beide Richtungen, asymmetrisch ein öffentlich/privates Schlüsselpaar. 2. Weil ein Hash sich nicht zurückrechnen lässt, eine Verschlüsselung mit dem passenden Schlüssel aber schon. 3. Linux Unified Key Setup, Standard für Festplatten-/Partitionsverschlüsselung unter Linux. 4. Asymmetrisch für den anfänglichen

Schlüsselaustausch, symmetrisch für die anschließende, schnellere Datenübertragung. 5. Nein, nicht wirksam: Im entsperrten Zustand ist die Partition für das laufende System normal lesbar, LUKS schützt nur den ausgeschalteten oder ausgehängten Zustand des Datenträgers.

10. Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] symmetrische, asymmetrische Verschlüsselung und Hashing klar voneinander abgrenzen kannst,
- [ ] eine LUKS-verschlüsselte Testpartition angelegt, eingebunden und wieder verschlossen hast,
- [ ] eine Datei mit GPG erfolgreich verschlüsselt und entschlüsselt hast,
- [ ] erklären kannst, warum Verschlüsselung ein entsperrtes, laufendes System nicht schützt.

Abschlussprojekt Band 1: Sicherer Debian-Grundserver

Installiere einen frischen Debian-13-Server und implementiere ein grundlegendes Sicherheitskonzept, das alle zehn Kapitel dieses Bandes zusammenführt:

1. **Benutzerkonzept:** mindestens ein administratives und ein reguläres Konto, keine Anmeldung als root (Kapitel 3-5).
2. **Rechtekonzept:** ein Projektverzeichnis mit Gruppenrechten und mindestens einer begründeten ACL (Kapitel 3).
3. **SSH-Konfiguration:** ausschließlich schlüsselbasierte Anmeldung, `PermitRootLogin no` (Kapitel 5).
4. **Firewall:** Standard-Deny mit expliziter SSH-Ausnahme, wahlweise über nftables oder ufw (Kapitel 7).
5. **Sicherheitsupdates:** `unattended-upgrades` auf das Security-Repository beschränkt (Kapitel 8).
6. **Dokumentation:** eine zusammenhängende, schriftliche Übersicht aller getroffenen Maßnahmen samt Begründung, als Grundlage für das in Band 2 vertiefte Hardening.

Dieses Projekt bildet den Ausgangspunkt für Band 2, in dem genau dieser Server systematisch weiter gehärtet wird.

Verzeichnis der Befehlsbeispiele

Beispiel V.1: Netzwerkarchitektur dieses Bandes … Seite 6
Beispiel V.2: server1 einrichten (Kurzübersicht) … Seite 7
Beispiel V.3: security-client vorbereiten … Seite 7
Beispiel 1.1: Installierte Pakete zählen … Seite 9
Beispiel 1.2: Laufende Netzwerkdienste auflisten … Seite 9
Beispiel 1.3: Aktuelle Debian-Version und Codename prüfen … Seite 9
Befehle zu Übung A … Seite 41
Vorgehen zu Übung B … Seite 37
Beispiel 2.1: Die grobe Architektur (Konzept) … Seite 12
Beispiel 2.2: Eine systemd-Service-Unit anzeigen … Seite 12
Beispiel 2.3: Rechte eines laufenden Prozesses anzeigen … Seite 13
Beispiel 2.4: Alle aktiven systemd-Units eines Typs auflisten … Seite 13
Beispiel 2.5: Eine eigene, einfache systemd-Unit anlegen … Seite 13
Beispiel 2.6: Inhalt von beispiel.service … Seite 13
Befehle zu Übung B … Seite 42
Beispiel 3.1: Einen Eintrag aus /etc/passwd lesen … Seite 16
Beispiel 3.2: Benutzer und Gruppe anlegen … Seite 17
Beispiel 3.3: Klassische Rechte setzen und lesen … Seite 17
Beispiel 3.4: Eine ACL ergänzen … Seite 17
Beispiel 3.5: Eine einfache sudoers-Regel (mit visudo bearbeiten) … Seite 17
Beispiel 4.1: Grundstruktur einer PAM-Konfiguration (Auszug aus /etc/pam.d/common-auth) … Seite 20
Beispiel 4.2: pam_pwquality installieren und konfigurieren … Seite 21
Beispiel 4.3: Auszug aus pwquality.conf … Seite 21
Beispiel 4.4: pam_faillock aktivieren (Auszug aus /etc/pam.d/common-auth) … Seite 21
Beispiel 4.5: faillock-Verhalten konfigurieren (/etc/security/faillock.conf) … Seite 21
Beispiel 5.1: Der Ablauf einer schlüsselbasierten Anmeldung (Konzept) … Seite 24
Beispiel 5.2: Ein Schlüsselpaar erzeugen (auf security-client) … Seite 24
Beispiel 5.3: Öffentlichen Schlüssel auf den Server übertragen … Seite 25
Beispiel 5.4: Zentrale sshd_config-Parameter … Seite 25
Beispiel 5.5: Konfiguration übernehmen … Seite 25
Beispiel 6.1: Offene Ports und zugehörige Prozesse anzeigen … Seite 28
Beispiel 6.2: Netzwerkschnittstellen und Routing anzeigen … Seite 29
Beispiel 6.3: Eine Netzwerkschnittstelle über netplan/interfaces konfigurieren (Debian-Standard: /etc/network/interfaces) … Seite 29
Beispiel 6.4: DNS-Auflösung testen … Seite 29
Beispiel 7.1: Grundstruktur einer nftables-Regelbasis (Konzept) … Seite 32
Beispiel 7.2: nftables installieren und eine Grundregelbasis einrichten … Seite 33
Beispiel 7.3: Inhalt von /etc/nftables.conf … Seite 33
Beispiel 7.4: Regeln aktivieren … Seite 33
Beispiel 7.5: Dieselbe Regel mit ufw … Seite 33
Beispiel 8.1: Grundlegende APT-Befehle … Seite 36
Beispiel 8.2: unattended-upgrades installieren … Seite 36
Beispiel 8.3: Zentrale Konfiguration (Auszug aus /etc/apt/apt.conf.d/50unattended-upgrades) … Seite 37
Beispiel 8.4: Trockenlauf der automatischen Aktualisierung … Seite 37
Beispiel 9.1: Protokolle mit journalctl durchsuchen … Seite 40
Beispiel 9.2: Fehlgeschlagene Anmeldeversuche finden … Seite 40
Beispiel 9.3: logrotate für ein eigenes Anwendungslog einrichten … Seite 40
Beispiel 9.4: Inhalt der logrotate-Konfiguration … Seite 41
Beispiel 10.1: TLS-Grundprinzip (stark vereinfacht) … Seite 44
Beispiel 10.2: Eine LUKS-verschlüsselte Partition anlegen (Testpartition oder Datei-Image) … Seite 45
Beispiel 10.3: Die Partition wieder aushängen und schließen … Seite 45
Beispiel 10.4: Ein GPG-Schlüsselpaar erzeugen und eine Datei verschlüsseln … Seite 45

Index

ACL – 16, 17, 18, 19, 47
APT – 18, 25, 27, 30, 36, 37, 38, 39
CVE – 9, 10, 11, 36, 37, 38, 39
DSA – 9, 36, 37, 38, 39
Firewall – 8, 28, 29, 30, 31, 32, 34, 35
GPG – 44, 45, 47
Hashing – 46, 47
journald – 14, 40, 42, 43
LUKS – 44, 45, 46, 47
nftables – 6, 32, 33, 34, 35, 47
PAM – 7, 20, 21, 22, 23
pam_faillock – 20, 21, 22, 23
pam_pwquality – 20, 21, 22, 23
rsyslog – 40, 41, 42, 43
SSH – 7, 9, 10, 14, 16, 20, 22, 24
sudo – 14, 17, 18, 19, 20, 22, 26, 34
systemd – 12, 13, 14, 15, 40, 42, 43
ufw – 11, 32, 33, 34, 35, 47
unattended-upgrades – 36, 37, 38, 39, 47
Verschlüsselung – 6, 44, 46, 47

LINUX SECURITY BAND 1: GRUNDLAGEN

Vom Prinzip zur eigenen, abgesicherten Konfiguration

Band 1 einer dreibändigen Fachbuchreihe über Linux-Sicherheit unter Debian 13. Zehn Kapitel führen von der Systemarchitektur über Benutzer, Authentifizierung und SSH bis zu Firewall-Grundlagen, Paketverwaltung, Protokollierung und Verschlüsselung. Jedes Kapitel folgt derselben Struktur: Einführung, technische Grundlagen, Fachbegriffe, Praxis, Sicherheitsbetrachtung, zwei vollständige Laborübungen, typische Fehler, Zusammenfassung, Kontrollfragen und Lernzielkontrolle.

DAS ERWARTET DICH

- Systemarchitektur, Benutzer, Gruppen und Berechtigungen fundiert erklärt
- PAM-gestützte Authentifizierung: Passwort-Richtlinien und Kontosperrung
- SSH-Sicherheit und Firewall-Grundlagen mit nftables und ufw
- Sicherheitsupdates, Protokollierung und Verschlüsselung (LUKS, GPG)
- 20 vollständige Laborübungen auf Debian 13, mit Musterlösungen und Kontrollfragen

Für wen?

Leser:innen mit Grundkenntnissen der Linux-Kommandozeile, die systematisch verstehen wollen, warum Sicherheitsmaßnahmen notwendig sind. Grundlage für Band 2 (Hardening) und Band 3 (Analyse und Ethical Hacking).

49 Seiten · Stand Oktober 2026 · Band 1 von 3 · Debian 13 (Trixie)

Hinweis: Keine Rechts- oder Anlageberatung. Marken- und Produktnamen gehören ihren jeweiligen Inhabern.